Computational Statistics

Lecture notes with R, Lectures 1–14

Contents

Lecture 1. Introduction 6

1.1 What the course is about 6
1.1.1 Organisation and scope . . . 7

1.2 Software 7

1.3 Running R code 8

1.4 Getting help 8

Lecture 2. Data Visualisation 9

2.1 Data visualisation with ggplot2 9
2.1.1 The mpg data frame . . . 9
2.1.2 Creating a ggplot . . . 9

2.2 Aesthetic mappings 10

2.3 Facets 11

2.4 Geometric objects 11

2.5 Statistical transformations 12

2.6 Position adjustments 13

2.7 Coordinate systems 15

2.8 The layered grammar of graphics 15

2.9 Graphics for communication 16
2.9.1 Labels . . . 16
2.9.2 Annotations . . . 16
2.9.3 Scales, legends and zooming . . . 17
2.9.4 Themes and saving . . . 17

Lecture 3. Data Transformation 18

3.1 Why transform data 18
3.1.1 The five verbs . . . 18

3.2 Filter rows with filter() 18
3.2.1 Comparisons . . . 19
3.2.2 Logical operators . . . 19

3.2.3 Missing values . . . 20

3.3 Arrange rows with arrange() 20

3.4 Select columns with select() 21

3.5 Add new variables with mutate() 21
3.5.1 Useful creation functions . . . 21

3.6 Grouped summaries with summarise() 22
3.6.1 The pipe . . . 23
3.6.2 Missing values and counts . . . 24
3.6.3 Useful summary functions . . . 24

3.7 Grouped mutates and filters 25

Lecture 4. Exploratory Data Analysis 27

4.1 What EDA is 27

4.2 Variation 27
4.2.1 Typical values . . . 28
4.2.2 Subgroups . . . 29
4.2.3 Unusual values . . . 29

4.3 Missing values 30

4.4 Covariation 31
4.4.1 A categorical and a continuous variable . . . 31
4.4.2 Two categorical variables . . . 32
4.4.3 Two continuous variables . . . 33

4.5 Patterns and models 33

Lecture 5. Import and Tidy Data 36

5.1 Tibbles 36

5.2 Data import with readr 37
5.2.1 Parsing a vector . . . 37
5.2.2 Parsing a file . . . 39
5.2.3 Writing files and other formats . . . 39

5.3 Tidy data 40

5.4 Gathering and spreading 41

5.5 Separating and uniting 42

Lecture 6. Relational Data 44

6.1 Relational data 44
6.1.1 Keys . . . 44

6.2 Mutating joins **44**
6.2.1 Duplicate keys . . . 45
6.2.2 Defining the key columns . . . 46

6.3 Filtering joins **47**

6.4 Set operations **47**

Lecture 7. Strings **49**

7.1 String basics **49**
7.1.1 Length, combining, subsetting . . . 49

7.2 Regular expressions **50**
7.2.1 Basic matches . . . 50
7.2.2 Anchors and character classes . . . 50
7.2.3 Repetition . . . 51
7.2.4 Grouping and backreferences . . . 51

7.3 Tools **51**
7.3.1 Detecting matches . . . 51
7.3.2 Extracting matches . . . 52
7.3.3 Grouped matches . . . 52
7.3.4 Replacing and splitting . . . 53

Lecture 8. Factors, Dates and Times **54**

8.1 Factors **54**
8.1.1 Creating factors . . . 54
8.1.2 The General Social Survey . . . 54
8.1.3 Modifying factor order . . . 55
8.1.4 Modifying factor levels . . . 55

8.2 Dates and times **56**
8.2.1 Creating dates and date-times . . . 56
8.2.2 Date-time components . . . 57
8.2.3 Time spans . . . 58
8.2.4 Time zones . . . 59

Lecture 9. Pipes **60**

9.1 Why pipes **60**

9.2 When not to use the pipe **61**

9.3 Other tools from magrittr **61**

Lecture 10. Functions **62**

10.1 When to write a function **62**

10.2 Functions are for humans and computers **62**

10.3 Conditional execution **63**
10.3.1 Conditions . . . 63
10.3.2 Multiple conditions . . . 64
10.3.3 Code style . . . 64

10.4 Function arguments **64**
10.4.1 Checking values . . . 65
10.4.2 Dot-dot-dot . . . 65

10.5 Return values **65**

10.6 Environment **66**

Lecture 12. Iteration **67**

12.1 Iteration **67**

12.2 For loops **67**
12.2.1 Variations . . . 67

12.3 For loops versus functionals **69**

12.4 The map functions **69**
12.4.1 Shortcuts . . . 70
12.4.2 Base R . . . 70

12.5 Dealing with failure **71**

12.6 Mapping over multiple arguments **71**

12.7 Walk, reduce and accumulate **72**

Lecture 13. Model Basics **73**

13.1 Models and their purpose **73**

13.2 A simple model **73**

13.3 Visualising models **75**

13.4 Formulas and model families **76**
13.4.1 Categorical variables . . . 76
13.4.2 Interactions . . . 76
13.4.3 Transformations . . . 77

13.5 Missing values and other model families **78**

Lecture 14. Model Building **79**

14.1 Building models step by step **79**

14.2 Why are low-quality diamonds more expensive? **79**
14.2.1 Price and carat . . . 79

14.2.2 A more complicated model 81

14.3 What affects the number of daily flights? 81
14.3.1 Day of week 82
14.3.2 Seasonal Saturday effect 83
14.3.3 Time of year: an alternative approach 84

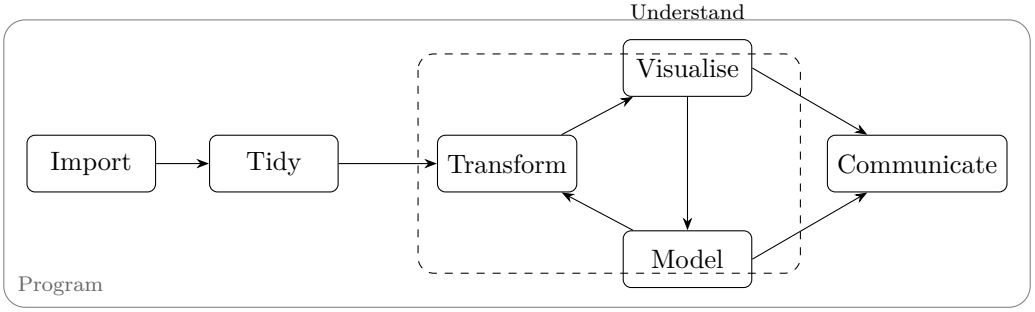
Lecture 1. Introduction

These notes follow the course slides, which are based on the first edition of *R for Data Science* (Wickham and Grolemund). All R code shown was run when the notes were compiled; lines starting with `##` are R's output. Material marked [Supplement] is not on the slides.

1.1 What the course is about

Motivation. Statistics courses usually start from clean data and a model. In practice most of the work happens before that: getting the data into R, cleaning it, and looking at it until the right question becomes clear. This course is about that work, done with code so that every step can be repeated and checked.

The course teaches how to collect, process, analyse and present data with a programming language. These goals are often summarised as **data science**, a term without a precise definition. A typical project passes through the following stages.



- **Import**: read data stored in a file, a database or a web API into a data frame in R.
- **Tidy**: store the data in a consistent form in which *each column is a variable and each row is an observation.*
- **Transform**: narrow down to the observations of interest (one city, the last year), create new variables from existing ones (speed from distance and time), and compute summaries (counts, means). Tidying and transforming together are called **wrangling**.
- **Visualise**: a good plot shows things you did not expect and raises new questions. Plots need a human to read them, so they do not scale well.
- **Model**: once a question is precise, a model answers it. Models are mathematical or computational tools and scale well, but every model makes assumptions that it cannot check itself.
- **Communicate**: results matter only if others can understand them.
- **Program**: programming runs through every stage. You need not be an expert programmer, but better programming lets you automate repeated tasks and solve new problems faster.

Visualising, transforming and modelling form a loop: a real analysis passes through it many times.

1.1.1 Organisation and scope

The course follows the book's order rather than the pipeline order, because the pipeline order is not the easiest way to learn the tools: visualisation and transformation first (immediate payoff), then wrangling, programming and modelling.

What is *not* covered: big data (we use small in-memory data sets; big data needs experience with small data first), languages other than R, non-rectangular data (images, sound, trees, text), and hypothesis *confirmation*. The course is about **hypothesis generation**: exploring data to find questions worth asking. Confirmatory inference is the subject of other statistics courses.

Generation versus confirmation [Supplement]

An observation can be used either to suggest a hypothesis or to test one, never both. If you search a data set for patterns and then test the pattern you found on the same data, the test is biased toward success. This is why exploration and confirmation are kept separate, a point that returns in Lecture 13.

1.2 Software

Motivation. The tools are R (the language), RStudio (the editor) and the tidyverse (a set of packages that share one design). Knowing how they fit together explains why the rest of the course looks the way it does.

R is downloaded from CRAN, the Comprehensive R Archive Network (the cloud mirror `https://cloud.r-project.org` picks a nearby server). **RStudio** is an integrated development environment for R: you type code in the console pane and press Enter to run it, or write scripts in the editor pane.

A **package** is a collection of functions, data and documentation that extends base R. The **tidyverse** is a family of packages built on a common philosophy of data and programming. Install it once, then load it in every session:

```
install.packages("tidyverse")
```

```
library(tidyverse)
```

Loading the tidyverse attaches its core packages, which are used in almost every analysis:

package	role
`ggplot2`	graphics (Lecture 2)
`dplyr`	data transformation (Lectures 3, 6)
`tidyr`	tidying (Lecture 5)
`readr`	reading rectangular files (Lecture 5)
`tibble`	a modern data frame (Lecture 5)
`stringr`, `forcats`	strings and factors (Lectures 7, 8)
`purrr`	functional programming (Lecture 12)

R also reports conflicts: `dplyr::filter()` masks `stats::filter()`, so after loading the tidyverse `filter()` means the dplyr version. The notation `package::function()` names a function from a specific package explicitly. Updates are installed with `tidyverse_update()`.

We also use three data packages outside the tidyverse:

```
install.packages(c("nycflights13", "gapminder", "Lahman"))
```

1.3 Running R code

Motivation. Everything else in the course is written as code, so it is worth fixing how code and its output are displayed.

In the console, R prints a prompt `>`, evaluates the line you type and prints the result:

```
1 + 2
## [1] 3
```

In these notes code is shown without the prompt, and output lines start with `##` so that they can be told apart from code (and copied back without harm, since `#` starts a comment). The `[1]` marks the index of the first element printed on that line: R's basic objects are vectors.

```
seq(10, 100, by = 5)
##  [1]  10  15  20  25  30  35  40  45  50  55  60  65  70  75  80  85  90  95
## [19] 100
```

Conventions used throughout: functions are written with parentheses, `sum()`, `mean()`; other objects (variables, data sets, arguments) without, `flights`, `x`; and `dplyr::mutate()` means the function `mutate()` from the package `dplyr`.

1.4 Getting help

Inside R, `?mean` opens the documentation of `mean()`, and `??regression` searches all help pages. Outside R, searching the exact error message (often with "R" added) usually finds someone who had the same problem; Stack Overflow and blogs collected at R-bloggers are the main sources.

When asking for help, give a **reproducible example** (reprex): the packages loaded, a small data set (`dput(head(df))` prints one as R code), and the shortest code that shows the problem. Preparing a reprex often reveals the answer.

```
dput(head(mtcars[, 1:3], 3))
## structure(list(mpg = c(21, 21, 22.8), cyl = c(6, 6, 4), disp = c(160,
## 160, 108)), row.names = c("Mazda RX4", "Mazda RX4 Wag", "Datsun 710"
## ), class = "data.frame")
```

Lecture 2. Data Visualisation

2.1 Data visualisation with ggplot2

Motivation. Visualisation is the best place to start programming with R because the payoff is immediate: a few lines of code turn a table nobody can read into a picture that answers a question. ggplot2 builds every plot from the same small set of parts, so learning those parts once covers almost every chart you will need.

Throughout we load the tidyverse, which attaches `ggplot2`:

```
library(tidyverse)
```

2.1.1 The mpg data frame

Do cars with big engines use more fuel than cars with small ones, and what does the relationship look like (positive or negative, linear or not)? The data frame `mpg` in ggplot2 (`ggplot2::mpg`) records 234 cars; `displ` is engine size in litres and `hwy` highway fuel efficiency in miles per gallon.

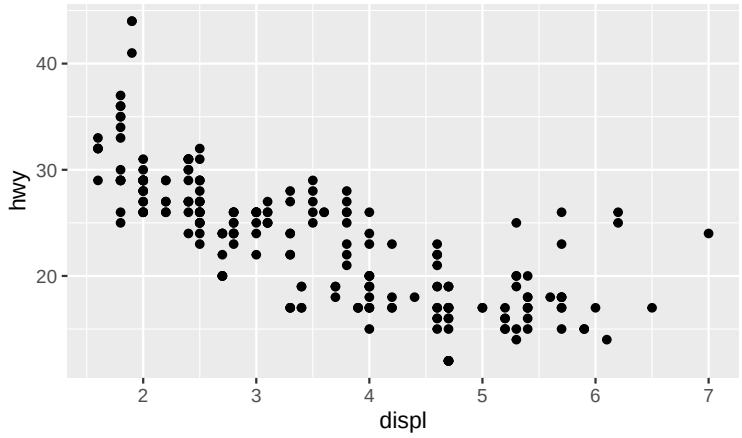
```
mpg
```

```
## # A tibble: 234 x 11
##    manufacturer model      displ  year   cyl trans      drv     cty   hwy fl    class
##    <chr>        <chr>      <dbl> <int> <int> <chr>      <chr> <int> <int> <chr> <chr>
##  1 audi         a4           1.8  1999     4 auto(l5)   f        18    29 p     compact
##  2 audi         a4           1.8  1999     4 manual(m5) f        21    29 p     compact
##  3 audi         a4           2    2008     4 manual(m6) f        20    31 p     compact
##  4 audi         a4           2    2008     4 auto(av)   f        21    30 p     compact
##  5 audi         a4           2.8  1999     6 auto(l5)   f        16    26 p     compact
##  6 audi         a4           2.8  1999     6 manual(m5) f        18    26 p     compact
##  7 audi         a4           3.1  2008     6 auto(av)   f        18    27 p     compact
##  8 audi         a4 quattro   1.8  1999     4 manual(m5) 4        18    26 p     compact
##  9 audi         a4 quattro   1.8  1999     4 auto(l5)   4        16    25 p     compact
## 10 audi         a4 quattro   2    2008     4 manual(m6) 4        20    28 p     compact
## # i 224 more rows
```

A **data frame** is a rectangular collection of variables (columns) and observations (rows); `mpg` is a tibble, the tidyverse version of a data frame (Lecture 5).

2.1.2 Creating a ggplot

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy))
```



The plot shows a negative relationship: bigger engines, lower efficiency. The code has three parts.

- `ggplot(data = mpg)` creates an empty coordinate system whose data set is `mpg`; on its own it draws a blank canvas.
- `geom_point()` adds a **layer** of points. Layers are added with `+`.
- The `mapping` argument says how variables are mapped to visual properties. It is always given by `aes()`; here `x = displ` and `y = hwy`. ggplot2 looks up `displ` and `hwy` inside the data set.

Every ggplot follows the **graphing template**

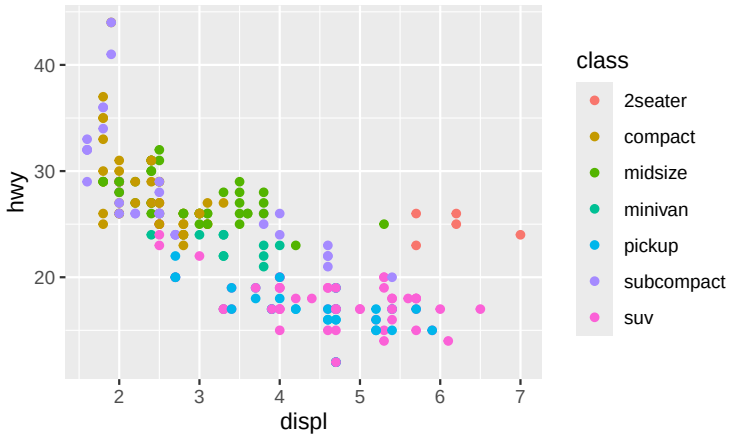
```
ggplot(data = <DATA>) +
  <GEOM_FUNCTION>(mapping = aes(<MAPPINGS>))
```

2.2 Aesthetic mappings

Motivation. The scatterplot leaves something unexplained: a group of cars with large engines but unusually high mileage. To explain them we need a third variable, and aesthetics are how a third (or fourth) variable enters a two-dimensional plot.

An **aesthetic** is a visual property of the objects in a plot: position, size, shape, colour, transparency. Mapping a variable to an aesthetic makes that property vary with the variable. Mapping `class` to colour reveals that the unusual cars are two-seaters (sports cars with light bodies):

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy, color = class))
```



ggplot2 assigns a level of the aesthetic to each value (**scaling**) and adds a legend explaining the mapping. Other aesthetics are used in the same way:

```
ggplot(data = mpg) + geom_point(mapping = aes(x = displ, y = hwy, size = class))
ggplot(data = mpg) + geom_point(mapping = aes(x = displ, y = hwy, alpha = class))
ggplot(data = mpg) + geom_point(mapping = aes(x = displ, y = hwy, shape = class))
```

- `size` and `alpha` (transparency) imply an order, so mapping an unordered variable such as `class` to them is not advised; ggplot2 warns about it.
- `shape` uses at most six shapes at a time; further groups are silently dropped (here SUVs disappear). R has 25 built-in shapes, numbered 0–24; shapes 21–24 have both a border (`colour`) and an interior (`fill`).

Setting versus mapping. To give *all* points the same property, set the aesthetic as an argument of the geom, *outside* `aes()`:

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy), color = "blue")
```

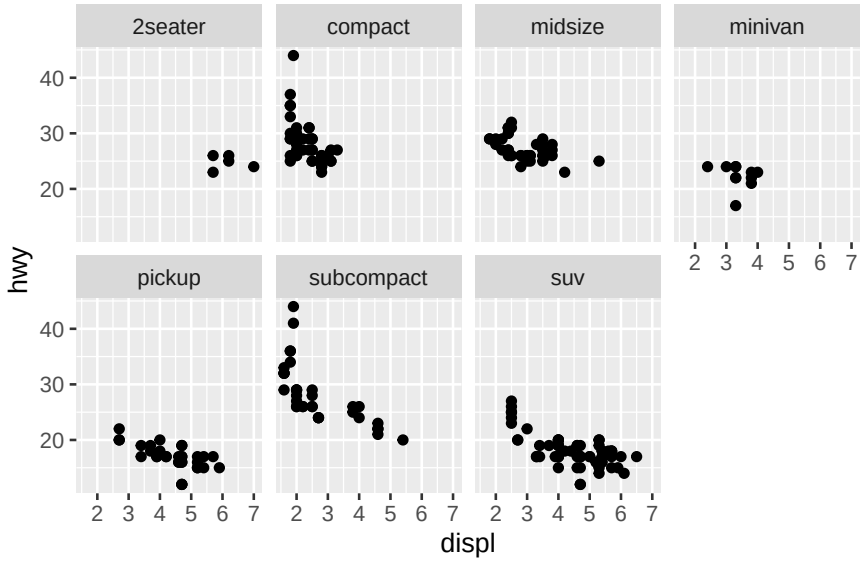
Writing `aes(color = "blue")` instead would map the constant string `"blue"` to colour, producing one group called "blue" drawn in the default first colour (red) with a legend.

2.3 Facets

Motivation. Aesthetics crowd a plot when the extra variable has many levels. The alternative is to draw one small plot per level and compare them side by side.

`facet_wrap()` splits the plot by one discrete variable, given as a formula:

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy)) +
  facet_wrap(~ class, nrow = 2)
```



`facet_grid(drv ~ cyl)` uses two variables, rows by the first and columns by the second; `facet_grid(. ~ cyl)` facets by columns only. Facets make each group easy to see on its own, at the cost of harder comparisons between groups than colour gives.

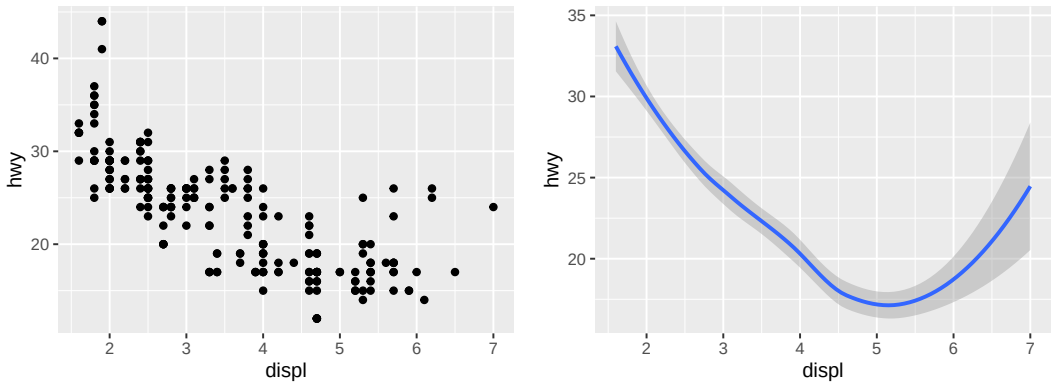
```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy)) +
  facet_grid(drv ~ class)
```

2.4 Geometric objects

Motivation. The same data can be drawn as points, as a smooth curve, as bars or as boxes. In ggplot2 this choice is a single function, the geom, while the data and mapping stay the same.

A **geom** is the geometrical object a plot uses to represent data. The two plots below use the same data and mapping but different geoms:

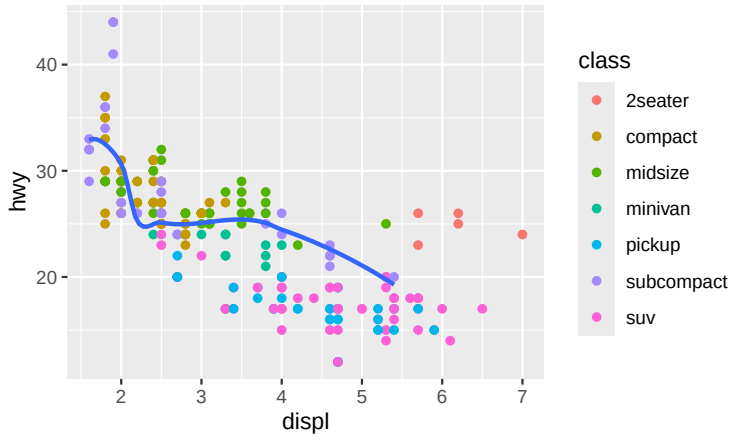
```
ggplot(data = mpg) + geom_point(mapping = aes(x = displ, y = hwy))
ggplot(data = mpg) + geom_smooth(mapping = aes(x = displ, y = hwy))
```



Every geom takes a `mapping`, but not every aesthetic applies to every geom: lines have a `linetype`, points do not. `geom_smooth(aes(linetype = drv))` draws one curve per drive train with different line types, and `aes(color = drv)` with different colours. Grouping happens automatically whenever a discrete variable is mapped; `aes(group = drv)` groups without adding a legend.

Several geoms can be layered. Mappings given in `ggplot()` are **global** and apply to every layer; mappings given in a geom are **local** to that layer and extend or override the global ones. A layer can even use its own data:

```
ggplot(data = mpg, mapping = aes(x = displ, y = hwy)) +
  geom_point(mapping = aes(color = class)) +
  geom_smooth(data = filter(mpg, class == "subcompact"), se = FALSE)
```



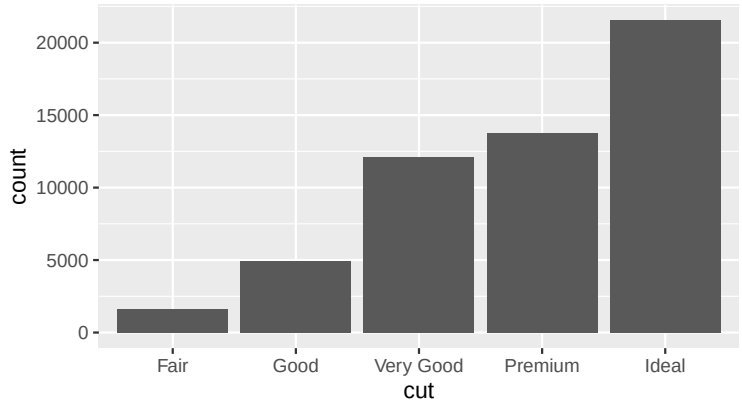
Here the smooth line is fitted to subcompact cars only (`filter()` is introduced in Lecture 3) and `se = FALSE` removes the confidence band. Without the `data` argument the line would be fitted to all cars.

2.5 Statistical transformations

Motivation. A bar chart of diamond cuts shows counts, yet the data contain no count column. Some plots draw values computed from the data rather than the data themselves; seeing where those values come from lets us change them.

The `diamonds` data set contains about 54 000 diamonds with their price, carat, cut, colour and clarity.

```
ggplot(data = diamonds) +
  geom_bar(mapping = aes(x = cut))
```

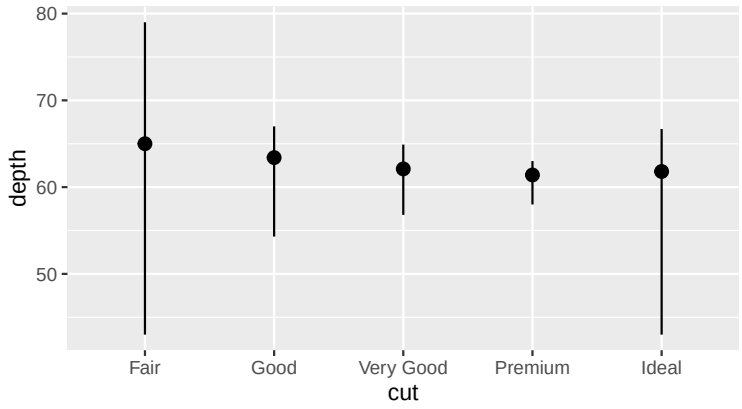


The y-axis, `count`, is not a variable of `diamonds`. Bar charts, histograms and frequency polygons bin the data and count; smoothers fit a model; boxplots compute summaries. The algorithm that computes the new values is a **stat**. The help page `?geom_bar` shows that its default stat is `"count"`, i.e. `stat_count()`; conversely `stat_count()` has default geom `geom_bar()`, so `ggplot(diamonds) + stat_count(aes(x = cut))` draws the same plot.

Reasons to use a stat explicitly:

- to plot a different computed variable, e.g. proportions instead of counts: `aes(x = cut, y = after_stat(prop), group = 1)`. The `group = 1` is needed because otherwise each bar is its own group and every proportion equals 1;
- to plot a summary of a variable for each x, e.g. the median depth with its range:

```
ggplot(data = diamonds) +
  stat_summary(
    mapping = aes(x = cut, y = depth),
    fun.min = min, fun.max = max, fun = median
  )
```

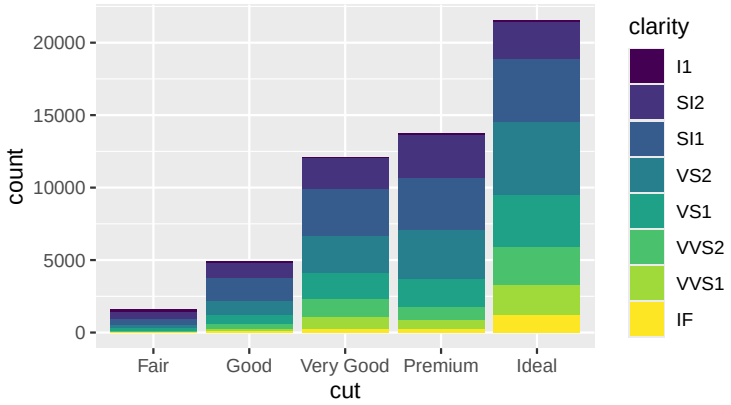


2.6 Position adjustments

Motivation. When bars or points share a position they overlap. How overlapping objects are arranged changes what the plot shows, so it is a deliberate choice rather than a default to accept.

Mapping a second variable to `fill` stacks the bars automatically:

```
ggplot(data = diamonds) +
  geom_bar(mapping = aes(x = cut, fill = clarity))
```



The heights of the coloured segments are the counts of each clarity within the cut, e.g. for ideal diamonds:

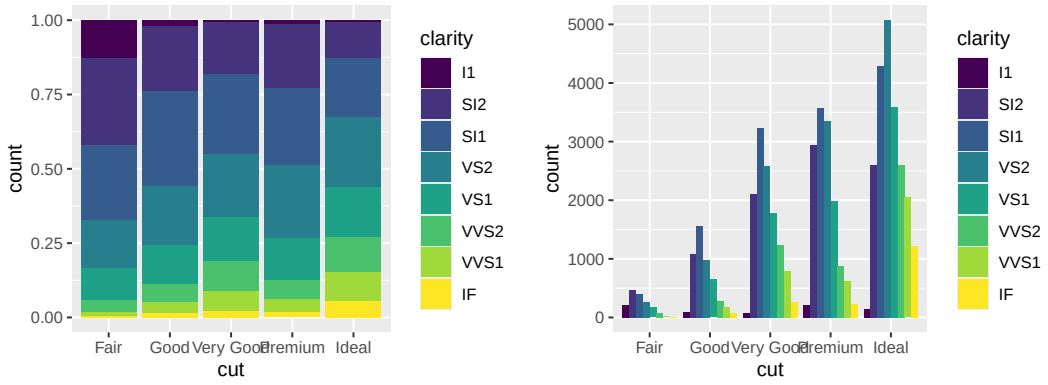
```
diamonds %>% filter(cut == "Ideal") %>% count(clarity)
```

```
## # A tibble: 8 x 2
##   clarity     n
##   <ord>   <int>
## 1 I1        146
## 2 SI2      2598
## 3 SI1      4282
## 4 VS2      5071
## 5 VS1      3589
## 6 VVS2     2606
## 7 VVS1     2047
## 8 IF       1212
```

The `position` argument controls the arrangement:

- `"stack"` (default for bars): segments on top of each other;
- `"identity"`: each bar starts at zero, so bars overlap (useful with `alpha` or for lines, not for bars);
- `"fill"`: stacked and rescaled to height 1, which compares *proportions* across groups;
- `"dodge"`: bars side by side, which compares individual counts.

```
ggplot(diamonds) + geom_bar(aes(x = cut, fill = clarity), position = "fill")
ggplot(diamonds) + geom_bar(aes(x = cut, fill = clarity), position = "dodge")
```



For scatterplots, `mpg` has 234 cars but the plot shows fewer points: the values are rounded, so many points coincide (**overplotting**). `position = "jitter"` adds a little random noise to each point.

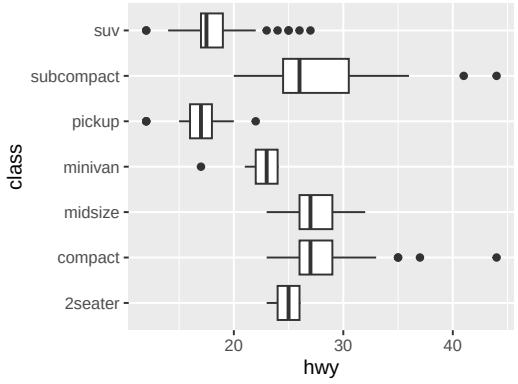
This makes the plot less accurate at small scales but more revealing at large scales; `geom_jitter()` is a shorthand.

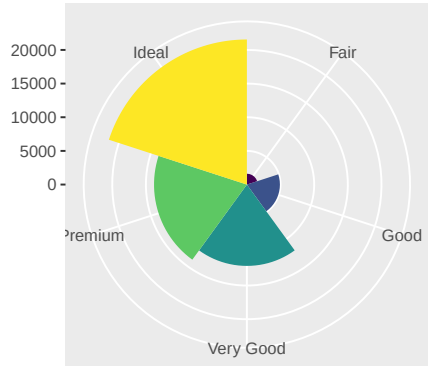
2.7 Coordinate systems

The default is Cartesian (`coord_cartesian()`, which also zooms with `xlim`, `ylim`). Others:

- `coord_flip()` swaps the axes, e.g. for horizontal boxplots with long labels;
- `coord_fixed(ratio = )` fixes the aspect ratio;
- `coord_quickmap()` sets the aspect ratio correctly for maps (`map_data("nz")` from the `maps` package gives map polygons for `geom_polygon()`);
- `coord_polar()` uses polar coordinates, which shows that a bar chart and a pie (Coxcomb) chart are the same graph in different coordinates.

```
ggplot(data = mpg, mapping = aes(x = class, y = hwy)) +
  geom_boxplot() + coord_flip()
bar <- ggplot(data = diamonds) +
  geom_bar(aes(x = cut, fill = cut), show.legend = FALSE, width = 1) +
  theme(aspect.ratio = 1) + labs(x = NULL, y = NULL)
bar + coord_polar()
```





2.8 The layered grammar of graphics

Motivation. All the pieces seen so far fit into one template. Knowing the template means any plot can be described by filling in its slots.

```
ggplot(data = <DATA>) +
  <GEOM_FUNCTION>(
     mapping = aes(<MAPPINGS>),
     stat = <STAT>,
     position = <POSITION>
  ) +
  <COORDINATE_FUNCTION> +
  <FACET_FUNCTION>
```

The seven parameters (data, mapping, geom, stat, position, coordinates, facets) describe every plot: start with a data set, transform it with a stat into the information to display, choose a geom to represent each value, map variables to aesthetics, place the geoms in a coordinate system and, optionally, split into facets.

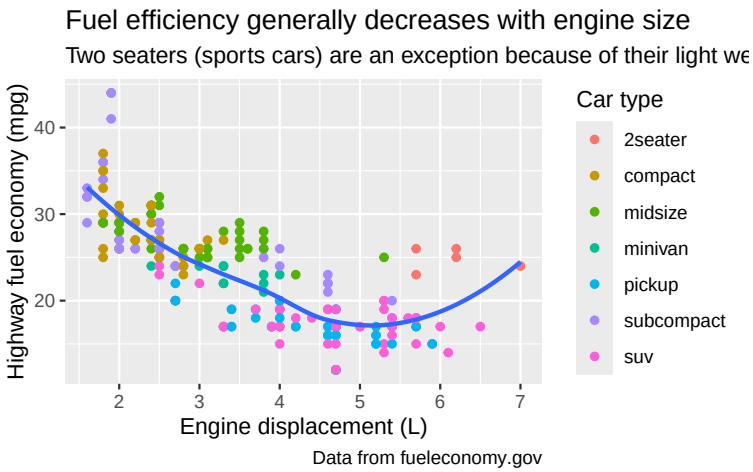
2.9 Graphics for communication

Motivation. Plots made during exploration are for you; plots in a report are for readers who do not know the data. Those need titles, readable labels and careful scales.

2.9.1 Labels

`labs()` sets the `title` (state the main finding, not just what is plotted), `subtitle`, `caption` (often the data source), axis labels and legend titles:

```
ggplot(mpg, aes(displ, hwy)) +
  geom_point(aes(color = class)) +
  geom_smooth(se = FALSE) +
  labs(
    title = "Fuel efficiency generally decreases with engine size",
    subtitle = "Two seaters (sports cars) are an exception because of their light weight
    ",
    caption = "Data from fueleconomy.gov",
    x = "Engine displacement (L)", y = "Highway fuel economy (mpg)", colour = "Car type"
  )
```



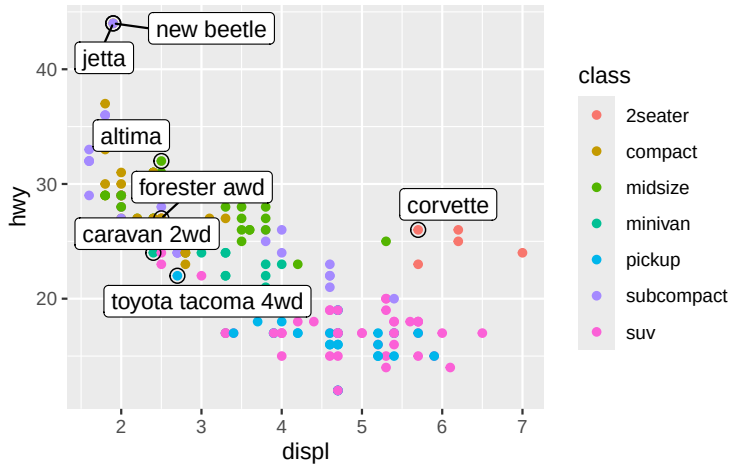
Axis labels can contain mathematical expressions written with `quote()`, e.g. `labs(x = quote(sum(x[i]^2, i == 1, n)))`; see `?plotmath`.

2.9.2 Annotations

To label individual observations, use `geom_text()` with its own data. For example, the most efficient car in each class:

```
best_in_class <- mpg %>%
  group_by(class) %>%
  filter(row_number(desc(hwy)) == 1)
```

```
ggplot(mpg, aes(displ, hwy)) +
  geom_point(aes(colour = class)) +
  geom_point(size = 3, shape = 1, data = best_in_class) +
  ggrepel::geom_label_repel(aes(label = model), data = best_in_class)
```



Plain `geom_text()` lets labels overlap each other and the points; `ggrepel::geom_label_repel()` moves them apart automatically.

2.9.3 Scales, legends and zooming

ggplot2 adds default scales behind the scenes: `scale_x_continuous()`, `scale_y_continuous()`, `scale_colour_discrete()`. Overriding them changes

- the **breaks** (tick positions): `scale_y_continuous(breaks = seq(15, 40, by = 5))`;
- the **labels**: `labels = NULL` removes them;
- the **transformation**: `scale_y_log10()`, `scale_x_reverse()`.

The legend position is set by `theme(legend.position = "bottom")` (or `"left"`, `"top"`, `"right"`, `"none"`).

To zoom there are two different operations:

- `coord_cartesian(xlim = c(5, 7), ylim = c(10, 30))` zooms the view *without removing data*, so the smoother is still fitted to all cars;
- `xlim(5, 7)`, `scale_x_continuous(limits = c(5, 7))`, or filtering the data first remove the points outside the range, so the smoother changes.

2.9.4 Themes and saving

Themes change the non-data elements (background, grid lines, fonts): `theme_bw()`, `theme_classic()`, `theme_minimal()` and others; ggplot2 has eight built-in themes. `ggsave("my-plot.pdf")` saves the most recent plot (size taken from the current device unless `width` and `height` are given).

ggplot2 has more than 30 geoms, and extension packages add many more; the ggplot2 cheat sheet is the best overview.

Lecture 3. Data Transformation

3.1 Why transform data

Motivation. Data rarely arrive in exactly the form a plot or a model needs. We usually have to pick the relevant rows and columns, create new variables, reorder, or summarise. dplyr turns each of these into one verb, and chaining a few verbs answers most descriptive questions.

Visualisation generates insight, but the data first need new variables, summaries, renamed columns or reordered rows. This lecture uses `dplyr` (core tidyverse) and the `flights` data from `nycflights13`: all 336 776 flights that departed New York City in 2013.

```
library(nycflights13)
library(tidyverse)
```

```
flights
```

```
## # A tibble: 336,776 x 19
##    year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time arr_delay
##   <int> <int> <int>    <int>          <int>     <dbl>    <int>          <int>     <dbl>
## 1  2013     1     1      517            515         2      830            819        11
## 2  2013     1     1      533            529         4      850            830        20
## 3  2013     1     1      542            540         2      923            850        33
## 4  2013     1     1      544            545        -1     1004           1022       -18
## 5  2013     1     1      554            600        -6      812            837       -25
## # i 336,771 more rows
## # i 10 more variables: carrier <chr>, flight <int>, tailnum <chr>, origin <chr>,
## #   dest <chr>, air_time <dbl>, distance <dbl>, hour <dbl>, minute <dbl>,
## #   time_hour <dttm>
```

Under each column name the tibble shows its type: `int` (integer), `dbl` (double, real number), `chr` (character), `dttm` (date-time); later we also meet `lgl` (logical), `fctr` (factor) and `date`.

3.1.1 The five verbs

verb	action
`filter()`	pick observations (rows) by their values
`arrange()`	reorder rows
`select()`	pick variables (columns) by their names
`mutate()`	create new variables from existing ones
`summarise()`	collapse many values to a single summary

All of them work the same way: the first argument is a data frame, the next arguments say what to do using variable names without quotes, and the result is a new data frame. Combined with `group_by()`, which makes every verb act group by group, they form a small language for data manipulation.

3.2 Filter rows with filter()

Motivation. The first step of most questions is to restrict attention to the relevant cases. Doing this correctly needs care with comparisons of real numbers and with missing values.

```
jan1 <- filter(flights, month == 1, day == 1)
jan1
```

```
## # A tibble: 842 x 19
##    year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time arr_delay
##   <int> <int> <int>    <int>          <int>     <dbl>    <int>          <int>     <dbl>
## 1  2013     1     1      517            515         2      830            819        11
## 2  2013     1     1      533            529         4      850            830        20
## 3  2013     1     1      542            540         2      923            850        33
## 4  2013     1     1      544            545        -1     1004           1022       -18
## 5  2013     1     1      554            600        -6      812            837       -25
## # i 837 more rows
## # i 10 more variables: carrier <chr>, flight <int>, tailnum <chr>, origin <chr>,
## #   dest <chr>, air_time <dbl>, distance <dbl>, hour <dbl>, minute <dbl>,
## #   time_hour <dttm>
```

Arguments separated by commas are combined with "and". dplyr never modifies its input, so the result must be saved with `<-`; wrapping an assignment in parentheses, `(jan1 <- ...)`, also prints it.

3.2.1 Comparisons

The comparison operators are `>`, `>=`, `<`, `<=`, `!=` and `==`. The common mistake `filter(flights, month = 1)` uses assignment instead of equality; dplyr stops with an error that suggests `month == 1`.

Floating-point numbers are stored with finite precision, so every real number you see is an approximation:

```
sqrt(2) ^ 2 == 2
```

```
## [1] FALSE
```

```
1 / 49 * 49 == 1
```

```
## [1] FALSE
```

```
near(sqrt(2) ^ 2, 2)
```

```
## [1] TRUE
```

Use `near()`, which compares up to a small tolerance, instead of `==` for real numbers.

Why $1/49 \cdot 49 \neq 1$ [Supplement]

A double has a 53-bit binary mantissa, so $1/49$ is rounded to the nearest representable number. Multiplying back by 49 carries this rounding error, and the result differs from 1 in the last bit:

```
print(1 / 49 * 49, digits = 17)
```

```
## [1] 0.99999999999999989
```

`near(x, y)` returns `abs(x - y) < tol` with `tol` about 1.5×10^{-8} (the square root of machine epsilon).

3.2.2 Logical operators

`&` is "and", `|` is "or", `!` is "not"; `xor()` is exclusive or. Flights in November or December:

```
filter(flights, month == 11 | month == 12)
filter(flights, month %in% c(11, 12))      # the same, shorter
```

`filter(flights, month == 11 | 12)` does *not* work: it means `(month == 11) | 12`, and `12` is `TRUE`, so every flight is kept. De Morgan's laws, $!(x \,\&\, y) = !x \,|\, !y$ and $!(x \,|\, y) = !x \,\&\, !y$, give two ways

of writing "delayed at most two hours both on arrival and departure":

```
filter(flights, !(arr_delay > 120 | dep_delay > 120))
filter(flights, arr_delay <= 120, dep_delay <= 120)
```

For complicated conditions, create them as explicit variables first so they can be checked. Do not use `&&` and `||` here: they handle only a single value and are meant for `if` statements (Lecture 10).

3.2.3 Missing values

`NA` ("not available") is an unknown value, and unknowns are contagious: almost any operation involving one is unknown.

```
NA > 5
```

```
## [1] NA
```

```
NA + 10
```

```
## [1] NA
```

```
NA == NA
```

```
## [1] NA
```

The last one makes sense with context: if Mary's age `x` and John's age `y` are both unknown, we cannot know whether they are the same age, so `x == y` is `NA`. To test for missingness use `is.na()`.

`filter()` keeps only rows where the condition is `TRUE`; it drops both `FALSE` and `NA`. To keep missing values, ask for them explicitly:

```
df <- tibble(x = c(1, NA, 3))
filter(df, x > 1)
```

```
## # A tibble: 1 x 1
##       x
##   <dbl>
## 1     3
```

```
filter(df, is.na(x) | x > 1)
```

```
## # A tibble: 2 x 1
##       x
##   <dbl>
## 1    NA
## 2     3
```

3.3 Arrange rows with arrange()

`arrange()` sorts rows by one or more columns, later columns breaking ties in earlier ones; `desc()` sorts in descending order. Missing values always go to the end, in either direction.

```
arrange(flights, desc(dep_delay))
```

```
## # A tibble: 336,776 x 19
##    year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time arr_delay
##   <int> <int> <int>    <int>          <int>     <dbl>    <int>          <int>     <dbl>
## 1  2013     1     9      641            900      1301     1242           1530      1272
## 2  2013     6    15     1432           1935      1137     1607           2120      1127
## 3  2013     1    10     1121           1635      1126     1239           1810      1109
## 4  2013     9    20     1139           1845      1014     1457           2210      1007
## 5  2013     7    22      845           1600      1005     1044           1815       989
## # i 336,771 more rows
## # i 10 more variables: carrier <chr>, flight <int>, tailnum <chr>, origin <chr>,
## #   dest <chr>, air_time <dbl>, distance <dbl>, hour <dbl>, minute <dbl>,
```

```
## #   time_hour <dttm>
```

```
df <- tibble(x = c(5, 2, NA))
arrange(df, desc(x))
```

```
## # A tibble: 3 x 1
##       x
##   <dbl>
## 1     5
## 2     2
## 3    NA
```

3.4 Select columns with select()

Motivation. Data sets with hundreds of variables are common, and the first job is often to narrow down to the few that matter.

```
select(flights, year, month, day)          # by name
select(flights, year:day)                  # all columns between year and day
select(flights, -(year:day))               # all except those
select(flights, seq(1, 10, by = 2))        # by position
select(flights, time_hour, air_time, everything())   # move to the front
rename(flights, tail_num = tailnum)        # rename, keep all others
```

Helper functions select by name pattern: `starts_with("abc")`, `ends_with("xyz")`, `contains("ijk")`, `matches("(.)\textbackslash \textbackslash 1")` (a regular expression, Lecture 7) and `num_range("x", 1:3)` for `x1`, `x2`, `x3`.

3.5 Add new variables with mutate()

Motivation. Many questions are about quantities that are not recorded directly, such as the speed of a flight or the time it made up in the air. mutate() computes them from existing columns.

`mutate()` adds columns at the end of the data set; to see them, start from a narrower data set:

```
flights_sml <- select(flights, year:day, ends_with("delay"), distance, air_time)
mutate(flights_sml,
  gain = dep_delay - arr_delay,
  hours = air_time / 60,
  gain_per_hour = gain / hours
)
```

```
## # A tibble: 336,776 x 10
##    year month   day dep_delay arr_delay distance air_time  gain hours gain_per_hour
##   <int> <int> <int>     <dbl>     <dbl>    <dbl>    <dbl> <dbl> <dbl>         <dbl>
## 1  2013     1     1         2        11     1400      227    -9  3.78         -2.38
## 2  2013     1     1         4        20     1416      227   -16  3.78         -4.23
## 3  2013     1     1         2        33     1089      160   -31  2.67        -11.6
## 4  2013     1     1        -1       -18     1576      183    17  3.05          5.57
## 5  2013     1     1        -6       -25      762      116    19  1.93          9.83
## # i 336,771 more rows
```

New columns can be used in the same call (`gain_per_hour` uses `gain`). `transmute()` keeps only the new variables.

3.5.1 Useful creation functions

The function must be **vectorised**: it takes a vector and returns a vector of the same length.

- Arithmetic `+ - * / ^`, with recycling of short arguments: `air_time / 60`, `x / sum(x)` (proportion), `y - mean(y)` (deviation).
- Modular arithmetic: `%/%` (integer division) and `%%` (remainder). Departure times are stored as `HHMM` numbers, so

```
transmute(flights, dep_time, hour = dep_time %/% 100, minute = dep_time %% 100)
```

```
## # A tibble: 336,776 x 3
##   dep_time  hour minute
##      <int> <dbl>  <dbl>
## 1      517     5     17
## 2      533     5     33
## 3      542     5     42
## 4      544     5     44
## 5      554     5     54
## # i 336,771 more rows
```

- Logs `log()`, `log2()`, `log10()`: useful for data over several orders of magnitude; `log2` is easiest to read (one unit = doubling).
- Offsets `lag()` and `lead()`, and cumulative aggregates `cumsum()`, `cumprod()`, `cummin()`, `cummax()`, `cummean()`:

```
x <- 1:10
lag(x)
```

```
##  [1] NA  1  2  3  4  5  6  7  8  9
```

```
cumsum(x)
```

```
##  [1]  1  3  6 10 15 21 28 36 45 55
```

```
cummean(x)
```

```
##  [1] 1.0 1.5 2.0 2.5 3.0 3.5 4.0 4.5 5.0 5.5
```

- Ranking: `min_rank()` gives ties the smallest rank (as in sports); `desc()` reverses. Variants: `row_number()`, `dense_rank()`, `percent_rank()`, `cume_dist()`, `ntile()`.

```
y <- c(1, 2, 2, NA, 3, 4)
min_rank(y)
```

```
## [1]  1  2  2 NA  4  5
```

```
min_rank(desc(y))
```

```
## [1]  5  3  3 NA  2  1
```

3.6 Grouped summaries with summarise()

Motivation. Summaries become informative when compared across groups: the average delay per day, per destination, per airline. group_by() plus summarise() is the workhorse for this.

`summarise()` collapses a data frame to one row:

```
summarise(flights, delay = mean(dep_delay, na.rm = TRUE))
```

```
## # A tibble: 1 x 1
##   delay
##   <dbl>
## 1  12.6
```

It becomes useful together with `group_by()`, which changes the unit of analysis from the whole data set to groups. Grouping does not change how the data look, only how the other verbs act on them:

```
by_day <- group_by(flights, year, month, day)
summarise(by_day, delay = mean(dep_delay, na.rm = TRUE))
```

```
## # A tibble: 365 x 4
## # Groups:   year, month [12]
##    year month   day delay
##   <int> <int> <int> <dbl>
## 1  2013     1     1 11.5
## 2  2013     1     2 13.9
## 3  2013     1     3 11.0
## 4  2013     1     4  8.95
## 5  2013     1     5  5.73
## # i 360 more rows
```

Each `summarise()` peels off one grouping level, which is what the message about grouped output means.

3.6.1 The pipe

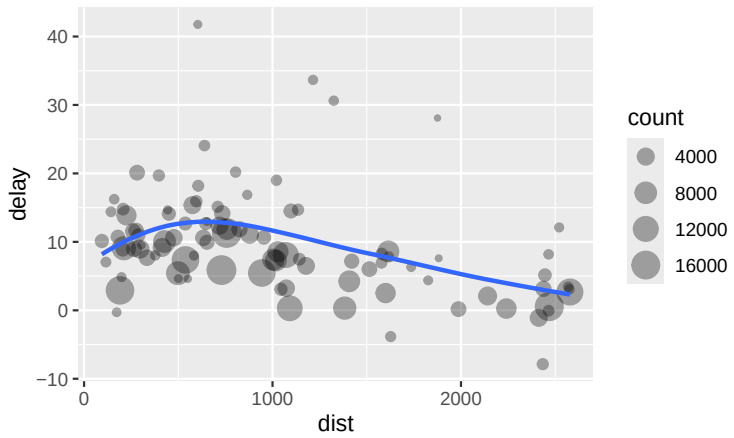
Relationship between distance and average delay for each destination, written with intermediate objects:

```
by_dest <- group_by(flights, dest)
delay <- summarise(by_dest, count = n(),
  dist = mean(distance, na.rm = TRUE), delay = mean(arr_delay, na.rm = TRUE))
delay <- filter(delay, count > 20, dest != "HNL")
```

Naming every intermediate result is tedious. The **pipe** `%>%` passes the result on the left as the first argument of the function on the right, so `x %>% f(y)` is `f(x, y)`; read it as "then":

```
delays <- flights %>%
  group_by(dest) %>%
  summarise(count = n(),
            dist = mean(distance, na.rm = TRUE),
            delay = mean(arr_delay, na.rm = TRUE)) %>%
  filter(count > 20, dest != "HNL")

ggplot(data = delays, mapping = aes(x = dist, y = delay)) +
  geom_point(aes(size = count), alpha = 1/3) +
  geom_smooth(se = FALSE)
```



Delays increase with distance up to about 750 miles and then decrease, perhaps because longer

flights can make up time in the air. ggplot2 layers are combined with `+`, not with the pipe (ggplot2 was written before the pipe).

3.6.2 Missing values and counts

Without `na.rm = TRUE` any missing value makes the summary `NA`. Missing delays here are cancelled flights, so it is cleaner to remove them once:

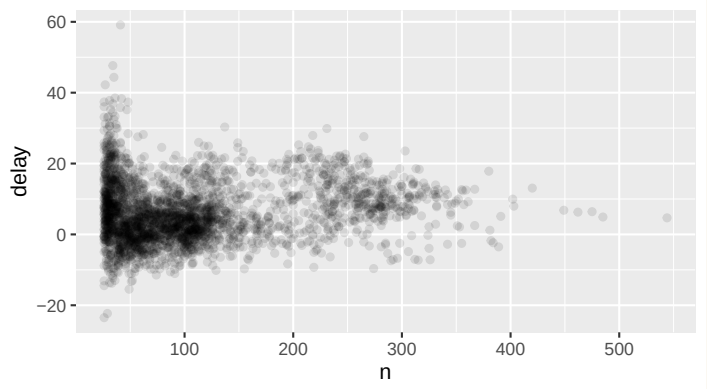
```
not_cancelled <- flights %>% filter(!is.na(dep_delay), !is.na(arr_delay))
```

Whenever you aggregate, include a count, `n()` or `sum(!is.na(x))`, so you do not draw conclusions from tiny groups: averages of few observations vary much more than averages of many.

Small groups have extreme averages [Supplement]

Plotting the average delay of each plane against its number of flights shows a funnel: planes with few flights have both the highest and the lowest averages. This is the variance σ^2/n of a mean at work, not a property of those planes. Filtering out small groups removes the noise:

```
not_cancelled %>%
  group_by(tailnum) %>%
  summarise(delay = mean(arr_delay), n = n()) %>%
  filter(n > 25) %>%
  ggplot(mapping = aes(x = n, y = delay)) +
  geom_point(alpha = 1/10)
```



3.6.3 Useful summary functions

- Location: `mean(x)`, `median(x)`; combined with logical subsetting, e.g. `mean(arr_delay[arr_delay > 0])` is the average positive delay.
- Spread: `sd(x)`, `IQR(x)`, `mad(x)`; the last two are robust to outliers.
- Rank: `min(x)`, `quantile(x, 0.25)`, `max(x)`.
- Position: `first(x)`, `nth(x, 2)`, `last(x)`.
- Counts: `n()`, `sum(!is.na(x))`, `n_distinct(x)`; `count(dest)` is a shortcut for grouping and counting, and `count(tailnum, wt = distance)` sums a weight (here total miles flown by each plane).
- Counts and proportions of logical values: `TRUE` counts as 1 and `FALSE` as 0, so `sum(x > 10)` is a count and `mean(y == 0)` a proportion.

```
not_cancelled %>%
  group_by(dest) %>%
  summarise(carriers = n_distinct(carrier)) %>%
  arrange(desc(carriers))
```

```
## # A tibble: 104 x 2
##   dest  carriers
##   <chr>    <int>
## 1 ATL          7
## 2 BOS          7
## 3 CLT          7
## 4 ORD          7
## 5 TPA          7
## # i 99 more rows
```

```
not_cancelled %>%
  group_by(year, month, day) %>%
  summarise(n_early = sum(dep_time < 500),      # flights before 5am
            hour_prop = mean(arr_delay > 60))   # share delayed over an hour
```

```
## # A tibble: 365 x 5
## # Groups:   year, month [12]
##    year month   day n_early hour_prop
##   <int> <int> <int>   <int>     <dbl>
## 1  2013     1     1       0    0.0722
## 2  2013     1     2       3    0.0851
## 3  2013     1     3       4    0.0567
## 4  2013     1     4       3    0.0396
## 5  2013     1     5       3    0.0349
## # i 360 more rows
```

Use `ungroup()` to remove grouping before operations that should act on the whole data set.

3.7 Grouped mutates and filters

Grouping also changes `filter()` and `mutate()`: functions such as `rank()`, `n()` and `mean()` are then computed within each group.

```
flights_sml %>%                          # the worst members of each group
  group_by(year, month, day) %>%
  filter(rank(desc(arr_delay)) < 10)
```

```
## # A tibble: 3,306 x 7
## # Groups:   year, month, day [365]
##    year month   day dep_delay arr_delay distance air_time
##   <int> <int> <int>     <dbl>     <dbl>    <dbl>    <dbl>
## 1  2013     1     1       853       851      184       41
## 2  2013     1     1       290       338     1134      213
## 3  2013     1     1       260       263      266       46
## 4  2013     1     1       157       174      213       60
## 5  2013     1     1       216       222      708      121
## # i 3,301 more rows
```

```
popular_dests <- flights %>%             # all groups bigger than a threshold
  group_by(dest) %>%
  filter(n() > 10000)
nrow(popular_dests)
```

```
## [1] 131440
```

```
popular_dests %>%                        # standardise to per-group metrics
  filter(arr_delay > 0) %>%
  mutate(prop_delay = arr_delay / sum(arr_delay)) %>%
  select(year:day, dest, arr_delay, prop_delay)
```

```
## # A tibble: 50,245 x 6
## # Groups:   dest [9]
##    year month   day dest  arr_delay prop_delay
##   <int> <int> <int> <chr>     <dbl>      <dbl>
## 1  2013     1     1 MIA          33  0.000235
## 2  2013     1     1 ORD          12  0.0000424
## 3  2013     1     1 FLL          19  0.0000938
## 4  2013     1     1 ORD           8  0.0000283
## 5  2013     1     1 LAX           7  0.0000344
## # i 50,240 more rows
```

Lecture 4. Exploratory Data Analysis

4.1 What EDA is

Motivation. The tools of Lectures 2 and 3 are now combined for their real purpose: understanding a data set before any formal analysis. EDA has no fixed procedure; it is a habit of asking questions and letting each answer suggest the next one.

The goal of **exploratory data analysis** (EDA) is to develop an understanding of your data. It is an iterative cycle:

1. generate questions about the data;
2. search for answers by visualising, transforming and modelling;
3. use what you learn to refine the questions or generate new ones.

Early on, investigate every idea; some will pan out and some will be dead ends. EDA also includes **data cleaning**: checking whether the data meet your expectations needs the same tools.

Two quotations set the tone: "There are no routine statistical questions, only questionable statistical routines" (D. R. Cox), and "Far better an approximate answer to the right question, which is often vague, than an exact answer to the wrong question, which can always be made precise" (J. W. Tukey). Two questions are always useful:

- What type of **variation** occurs *within* my variables?
- What type of **covariation** occurs *between* my variables?

Definitions. A *variable* is a quantity, quality or property you can measure; a *value* is its state when measured; an *observation* is a set of measurements made under similar conditions (usually on the same object at the same time), containing several values; *tabular data* is a set of values, each associated with a variable and an observation.

4.2 Variation

Motivation. Every variable varies, from measurement error or from real differences between objects, and the pattern of that variation is itself information. The way to see it is to plot the variable's distribution.

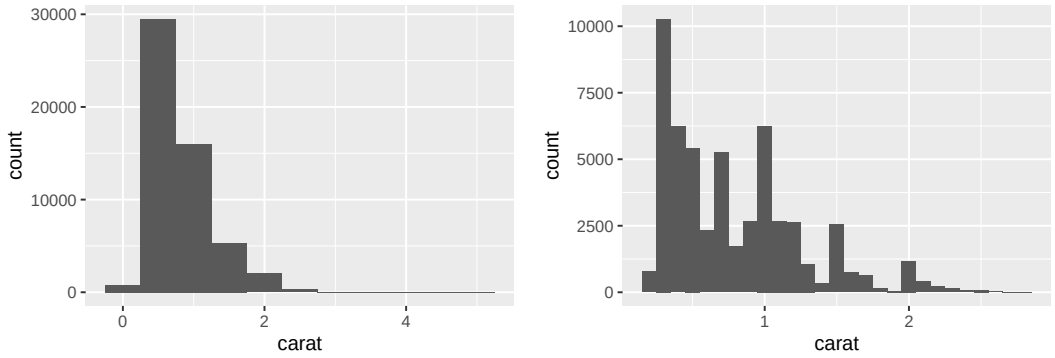
Variation is the tendency of the values of a variable to change from measurement to measurement. How to visualise a distribution depends on the type of variable.

Categorical variables take a small set of values; use a bar chart, whose heights are the counts:

```
ggplot(data = diamonds) + geom_bar(mapping = aes(x = cut))
diamonds %>% count(cut)                    # the same numbers
```

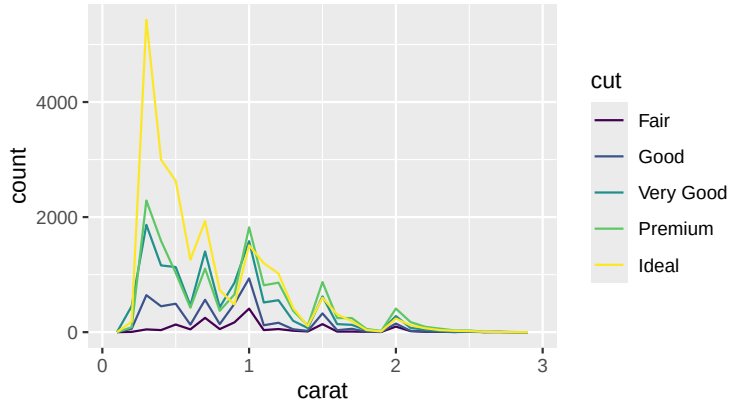
Continuous variables (numbers, date-times) take infinitely many ordered values; use a histogram, which counts the observations in equal-width bins:

```
ggplot(data = diamonds) +
  geom_histogram(mapping = aes(x = carat), binwidth = 0.5)
smaller <- diamonds %>% filter(carat < 3)
ggplot(data = smaller, mapping = aes(x = carat)) +
  geom_histogram(binwidth = 0.1)
```



`diamonds %>% count(cut_width(carat, 0.5))` computes the bars by hand. Always try several binwidths: different binwidths reveal different patterns. To overlay several distributions, `geom_freqpoly()` draws the counts as lines, which overlap better than bars:

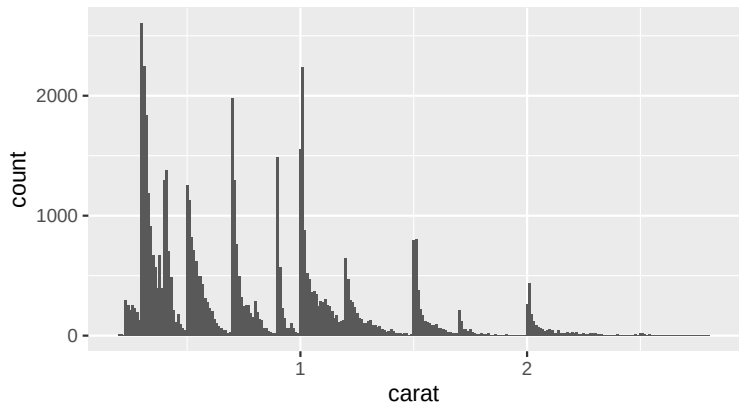
```
ggplot(data = smaller, mapping = aes(x = carat, colour = cut)) +
  geom_freqpoly(binwidth = 0.1)
```



4.2.1 Typical values

Turn the picture into questions: Which values are most common, and why? Which are rare, and does that match expectations? Are there unusual patterns? With a fine binwidth:

```
ggplot(data = smaller, mapping = aes(x = carat)) +
  geom_histogram(binwidth = 0.01)
```



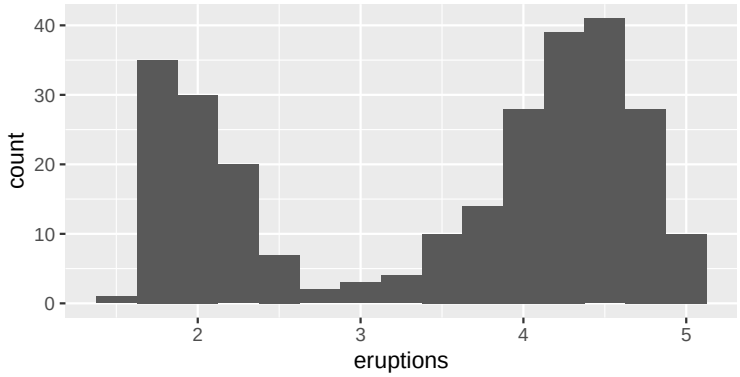
Why are there more diamonds at whole carats and common fractions? Why are there more diamonds

slightly to the right of each peak than to the left? Why none above 3 carats? (A plausible answer to the first two: cutters aim for round weights because buyers pay a premium for them [Supplement].)

4.2.2 Subgroups

Clusters of similar values suggest subgroups. Ask how observations within a cluster are similar, how clusters differ, how to explain them, and whether the clusters could be misleading. The eruption lengths of the Old Faithful geyser fall into short (about 2 minutes) and long (4–5 minutes) eruptions with little in between:

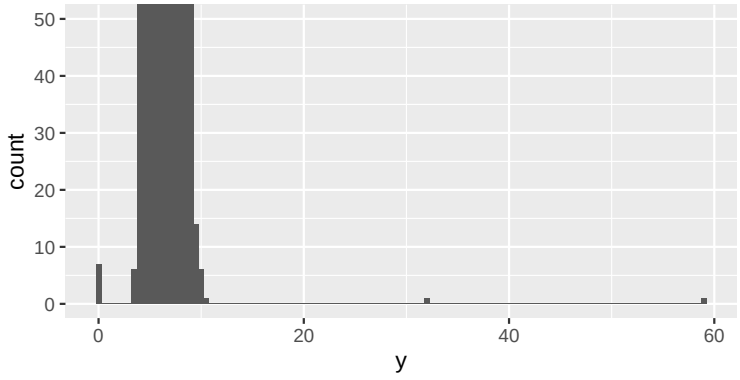
```
ggplot(data = faithful, mapping = aes(x = eruptions)) +
  geom_histogram(binwidth = 0.25)
```



4.2.3 Unusual values

The histogram of the width `y` of diamonds has a suspiciously wide x-axis. The rare values are there but their bars are too short to see; zooming with `coord_cartesian(ylim = c(0, 50))` reveals values near 0, 30 and 60 mm. (`coord_cartesian()` zooms without dropping data; `ylim()` would drop it.)

```
ggplot(diamonds) +
  geom_histogram(mapping = aes(x = y), binwidth = 0.5) +
  coord_cartesian(ylim = c(0, 50))
```




```
unusual <- diamonds %>%
  filter(y < 3 | y > 20) %>%
  select(price, x, y, z) %>%
  arrange(y)
unusual
```

```
## # A tibble: 9 x 4
##   price     x     y     z
##   <int> <dbl> <dbl> <dbl>
## 1  5139     0     0     0
## 2  6381     0     0     0
## 3 12800     0     0     0
## 4 15686     0     0     0
## 5 18034     0     0     0
## # i 4 more rows
```

A width of 0 mm is impossible, so those are errors (here all of `x`, `y`, `z` are 0 for some). Widths of 32 and 59 mm are implausible: such diamonds would be over an inch wide yet are priced like ordinary stones. Outliers are sometimes data-entry errors and sometimes important new science. Good practice:

- repeat the analysis with and without the outliers;
- if they have little effect and you cannot explain them, replace them with missing values and move on;
- if they have a substantial effect, do not drop them without justification: find the cause and disclose the removal.

4.3 Missing values

Motivation. After finding bad values we must decide what to do with them. Deleting whole rows throws away the good measurements in those rows too; marking only the bad value as missing keeps them.

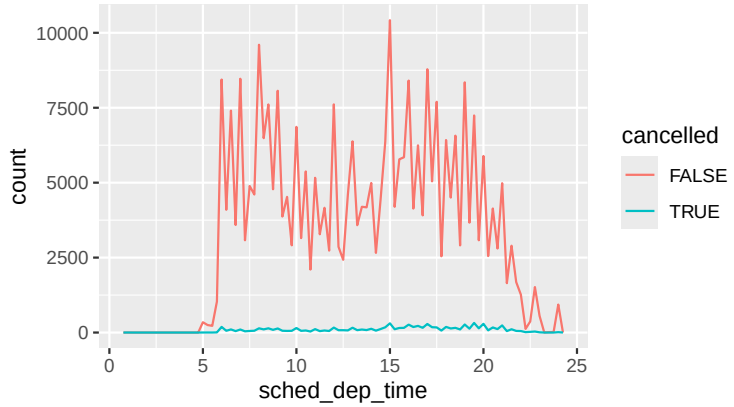
Rather than dropping rows (`filter(between(y, 3, 20))`), replace unusual values by `NA`:

```
diamonds2 <- diamonds %>%
  mutate(y = ifelse(y < 3 | y > 20, NA, y))
```

`ifelse(test, yes, no)` is vectorised: where the logical vector `test` is `TRUE` it returns the value of `yes`, elsewhere the value of `no`. (`case_when()` handles more than two cases.) ggplot2 does not plot missing values but warns that they were removed; `geom_point(na.rm = TRUE)` suppresses the warning.

Sometimes missingness itself is informative. In `flights`, a missing `dep_time` means a cancelled flight, so we can compare scheduled departure times of cancelled and non-cancelled flights:

```
flights %>%
  mutate(
    cancelled = is.na(dep_time),
    sched_hour = sched_dep_time %/% 100,
    sched_min = sched_dep_time %% 100,
    sched_dep_time = sched_hour + sched_min / 60
  ) %>%
  ggplot(mapping = aes(sched_dep_time)) +
  geom_freqpoly(mapping = aes(colour = cancelled), binwidth = 1/4)
```



The comparison is poor because cancelled flights are few; the next section shows how to compare groups of very different sizes.

4.4 Covariation

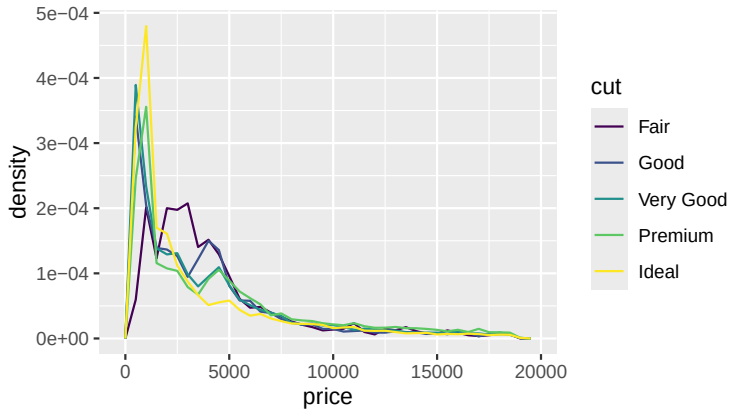
Motivation. Variation describes one variable; covariation describes how two vary together, which is where most interesting questions live. The right plot depends on the types of the two variables.

Covariation is the tendency of two or more variables to vary together in a related way. The best way to spot it is to visualise the relationship.

4.4.1 A categorical and a continuous variable

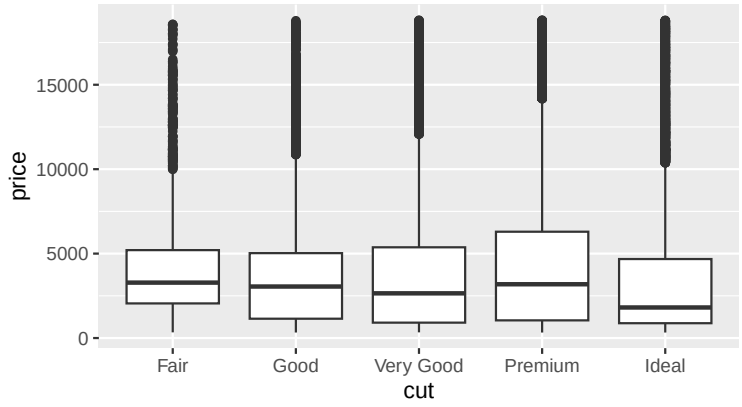
Frequency polygons of price by cut are hard to compare because the counts differ so much between cuts. Displaying **density** instead, the count standardised so that each polygon has area one, makes the shapes comparable:

```
ggplot(data = diamonds, mapping = aes(x = price, y = after_stat(density))) +
  geom_freqpoly(mapping = aes(colour = cut), binwidth = 500)
```



Surprisingly, fair diamonds (the lowest quality) seem to have the highest average price. A **boxplot** summarises each distribution compactly: the box spans the interquartile range (IQR) with a line at the median, whiskers extend to the farthest non-outlying points, and points more than 1.5×IQR beyond the box are drawn individually.

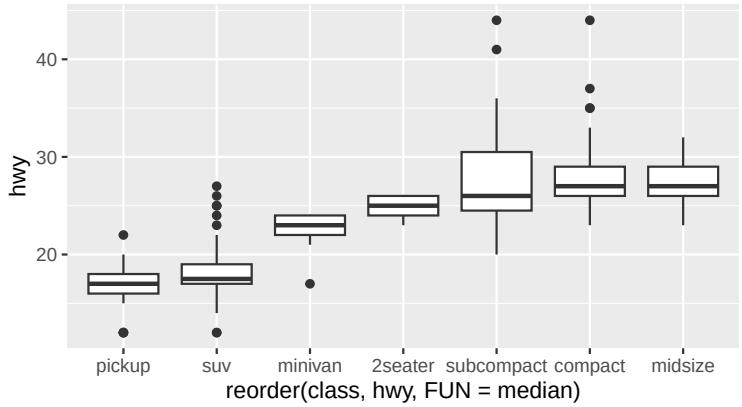
```
ggplot(data = diamonds, mapping = aes(x = cut, y = price)) +
  geom_boxplot()
```



It confirms the counterintuitive finding: better diamonds are cheaper on average. (Section 4.5 explains why.)

Categories without an intrinsic order are easier to read when reordered by a summary of the continuous variable:

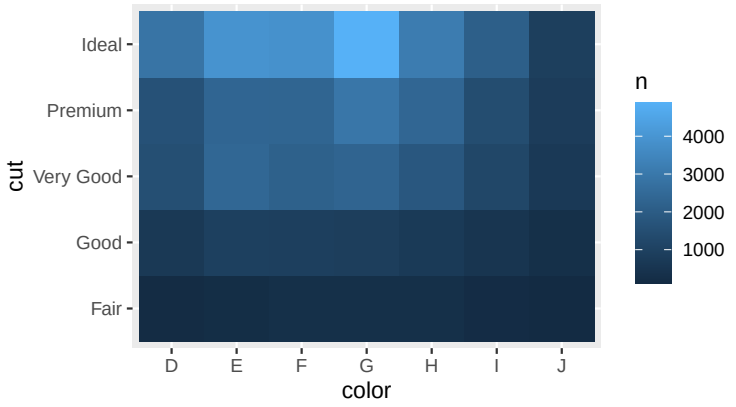
```
ggplot(data = mpg) +
  geom_boxplot(mapping = aes(x = reorder(class, hwy, FUN = median), y = hwy))
```



4.4.2 Two categorical variables

Count the observations for each combination, either with `geom_count()` (circle size proportional to count) or by computing the counts with dplyr and drawing a heat map:

```
diamonds %>%
  count(color, cut) %>%
  ggplot(mapping = aes(x = color, y = cut)) +
  geom_tile(mapping = aes(fill = n))
```

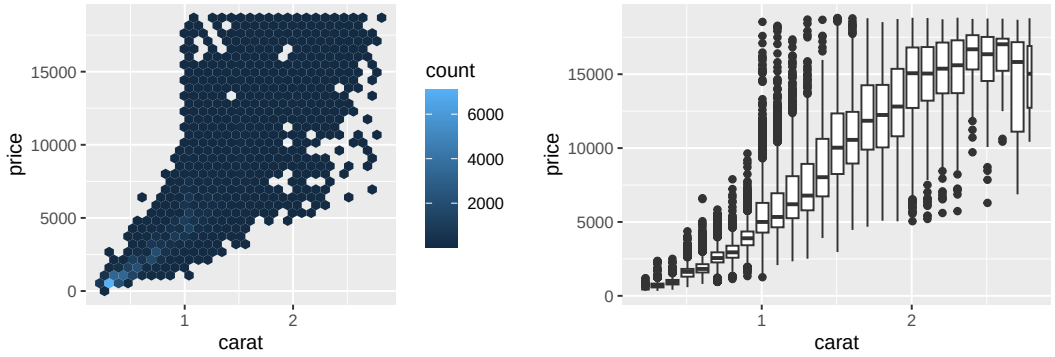


4.4.3 Two continuous variables

A scatterplot is the natural choice, but with 54 000 points it overplots. Remedies: transparency (`alpha = 1/100`), or binning in two dimensions with `geom_bin2d()` (rectangles) or `geom_hex()` (hexagons, needs `hexbin`), which colour each bin by its count. Another option bins one variable so it acts like a categorical one:

```
ggplot(data = smaller) +
  geom_hex(mapping = aes(x = carat, y = price))
ggplot(data = smaller, mapping = aes(x = carat, y = price)) +
  geom_boxplot(mapping = aes(group = cut_width(carat, 0.1)))
```

```
## Warning: Orientation is not uniquely specified when both the x and y aesthetics are continuous.
## Picking default orientation 'x'.
```



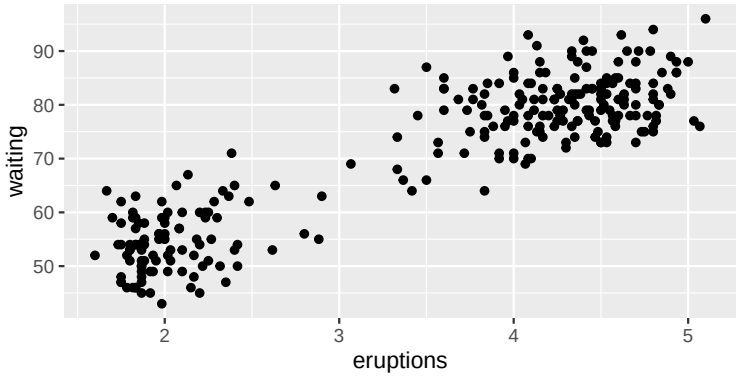
`cut_width(x, w)` makes bins of width `w`; `cut_number(x, n)` makes `n` bins with approximately the same number of points each.

4.5 Patterns and models

Motivation. A pattern suggests a relationship, but a strong relationship can hide weaker ones. A model lets us remove the strong one and look at what is left, which resolves the puzzle of the cheap good diamonds.

When a systematic relationship exists between two variables, it appears as a pattern. Then ask: could it be coincidence? How can the relationship be described, and how strong is it? What other variables might affect it? Does it change within subgroups? The Old Faithful scatterplot shows that longer waiting times go with longer eruptions, and also shows the two clusters:

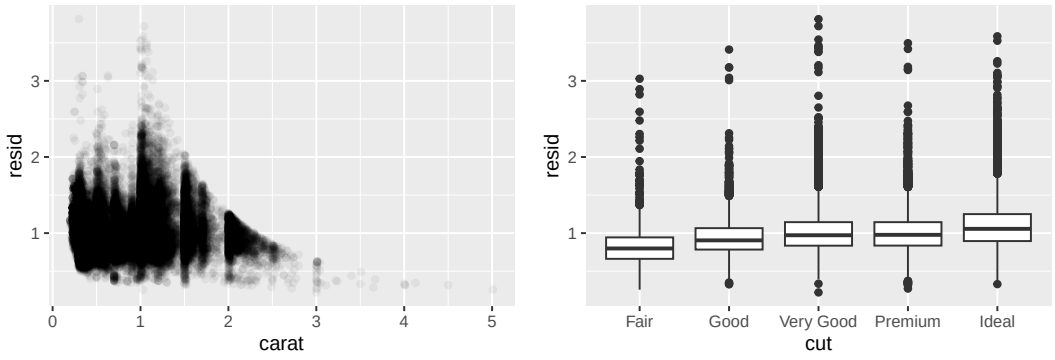
```
ggplot(data = faithful) +
  geom_point(mapping = aes(x = eruptions, y = waiting))
```



Patterns reveal covariation. Variation creates uncertainty; covariation reduces it, because knowing one variable improves prediction of the other. Only if the covariation is causal can one variable be used to *control* the other.

Models extract patterns. In `diamonds`, price and carat are tightly related, and cut is related to carat (better cuts tend to be smaller stones). To see the effect of cut alone, fit a model of price on carat and look at the residuals, the part of price the model does not explain:

```
mod <- lm(log(price) ~ log(carat), data = diamonds)
diamonds2 <- diamonds %>%
  add_residuals(mod) %>%
  mutate(resid = exp(resid))
ggplot(data = diamonds2) +
  geom_point(mapping = aes(x = carat, y = resid), alpha = 1/20)
ggplot(data = diamonds2) +
  geom_boxplot(mapping = aes(x = cut, y = resid))
```



Once the strong price–carat relationship is removed, the expected order appears: *relative to their size*, better diamonds are more expensive. Lectures 13 and 14 study models in detail.

Why log–log [Supplement]

The model $\log(\text{price}) = a_0 + a_1 \log(\text{carat})$ says $\text{price} = e^{a_0}\,\text{carat}^{a_1}$, a power law. On the log scale the relationship is linear with roughly constant spread, which a straight line fits well. The residual on the log scale is $\log(\text{price}/\widehat{\text{price}})$, so `exp(resid)` is the ratio of the actual price to the price predicted from carat alone: values above 1 mean more expensive than a typical diamond of that size.

```r
coef(mod)
```

```
## (Intercept)  log(carat)
##    8.448661    1.675817
```

Lecture 5. Import and Tidy Data

5.1 Tibbles

Motivation. Every tidyverse function takes and returns a tibble. Tibbles are data frames with a few old behaviours removed, and knowing those differences avoids surprises when moving between base R and the tidyverse.

A **tibble** is a data frame that does less: it never changes the type of inputs (strings are not turned into factors), never changes variable names, never creates row names, and complains when a variable does not exist.

```
as_tibble(iris)                          # data frame -> tibble
```

```
## # A tibble: 150 x 5
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
##          <dbl>       <dbl>        <dbl>       <dbl> <fct>
## 1          5.1         3.5          1.4         0.2 setosa
## 2          4.9         3            1.4         0.2 setosa
## 3          4.7         3.2          1.3         0.2 setosa
## 4          4.6         3.1          1.5         0.2 setosa
## 5          5           3.6          1.4         0.2 setosa
## # i 145 more rows
```

```
tibble(x = 1:5, y = 1, z = x ^ 2 + y)  # from vectors; y is recycled
```

```
## # A tibble: 5 x 3
##       x     y     z
##   <int> <dbl> <dbl>
## 1     1     1     2
## 2     2     1     5
## 3     3     1    10
## 4     4     1    17
## 5     5     1    26
```

```
tribble(                                 # "transposed tibble": written row by row
  ~x, ~y, ~z,
  "a", 2, 3.6,
  "b", 1, 8.5
)
```

```
## # A tibble: 2 x 3
##   x         y     z
##   <chr> <dbl> <dbl>
## 1 a         2   3.6
## 2 b         1   8.5
```

`as.data.frame()` converts back, for old functions that do not accept tibbles. Column names may be non-syntactic (e.g. `` `:)` ``, `` `2000` ``) if surrounded by backticks.

Printing. A tibble prints the first 10 rows and the columns that fit on screen, with each column's type. `print(n = 10, width = Inf)` shows more; the defaults are changed by `options(tibble.print_max = n, tibble.print_min = m)` (if there are more than `n` rows, print `m`) and `options(tibble.width = Inf)`.

Subsetting. `$` extracts a column by name, `[[ ]]` by name or position. Unlike data frames, tibbles never do partial matching of names and warn if a column does not exist. In a pipe, the placeholder `.` stands for the incoming object:

```
df <- tibble(x = runif(3), y = rnorm(3))
df$x
```

```
## [1] 0.08075014 0.83433304 0.60076089
```

```
df[[2]]
```
```
## [1] -1.00599730 -0.08433889 -0.55406480
```

```
df %>% .$x
```
```
## [1] 0.08075014 0.83433304 0.60076089
```

5.2 Data import with readr

Motivation. Analysing our own data starts with reading a file. A text file only contains characters; turning them into numbers, dates and factors is guesswork that occasionally goes wrong, so we need to see how readr guesses and how to correct it.

`readr` turns flat files into tibbles:

- `read_csv()` (comma-delimited), `read_csv2()` (semicolon-separated, where the comma is the decimal mark), `read_tsv()` (tab-delimited), `read_delim()` (any delimiter);
- `read_fwf()` (fixed-width), `read_table()` (columns separated by white space), `read_log()` (Apache-style logs).

They share their arguments, so learning `read_csv()` covers them all. The first argument is a path, e.g. `read_csv("data/heights.csv")`; an inline string is convenient for experiments:

```
read_csv("a,b,c
1,2,3
4,5,6")
```
```
## # A tibble: 2 x 3
##       a     b     c
##   <dbl> <dbl> <dbl>
## 1     1     2     3
## 2     4     5     6
```

Useful arguments:

```
read_csv("The first line of metadata
  The second line of metadata
  x,y,z
  1,2,3", skip = 2)                                  # skip n lines
read_csv("# A comment I want to skip
  x,y,z
  1,2,3", comment = "#")                             # drop comment lines
read_csv("1,2,3\n4,5,6", col_names = FALSE)          # no header: X1, X2, X3
read_csv("1,2,3\n4,5,6", col_names = c("x", "y", "z"))
read_csv("a,b,c\n1,2,.", na = ".")                   # symbol for missing values
```

Compared with base R's `read.csv()`, readr is faster, produces tibbles, does not convert strings to factors, and behaves the same on every operating system.

5.2.1 Parsing a vector

`parse_*()` functions turn a character vector into a more specialised vector; they are the building blocks of `read_csv()`.

```
str(parse_logical(c("TRUE", "FALSE", "NA")))
```
```
##  logi [1:3] TRUE FALSE NA
```

```
str(parse_date(c("2010-01-01", "1979-10-14")))
```

```
##  Date[1:2], format: "2010-01-01" "1979-10-14"
```

```
parse_integer(c("1", "231", ".", "456"), na = ".")
```

```
## [1]   1 231  NA 456
```

```
x <- parse_integer(c("123", "345", "abc", "123.45"))
```

```
## Warning: 2 parsing failures.
## row col               expected actual
##   3  -- no trailing characters abc
##   4  -- no trailing characters 123.45
```

```
x
```

```
## [1] 123 345  NA  NA
## attr(,"problems")
## # A tibble: 2 x 4
##     row   col expected               actual
##   <int> <int> <chr>                  <chr>
## 1     3    NA no trailing characters abc
## 2     4    NA no trailing characters 123.45
```

```
problems(x)
```

```
## # A tibble: 2 x 4
##     row   col expected               actual
##   <int> <int> <chr>                  <chr>
## 1     3    NA no trailing characters abc
## 2     4    NA no trailing characters 123.45
```

Failures become `NA` with a warning; `problems()` lists them.

Numbers. Three complications: the decimal mark differs between countries (`.` or `,`); numbers come with context (`"$1000"`, `"10%"`); and grouping marks vary (`1,000,000`, `1.000.000`, `1'000'000`).

```
parse_double("1,23", locale = locale(decimal_mark = ","))
```

```
## [1] 1.23
```

```
parse_number("It cost $123.45")
```

```
## [1] 123.45
```

```
parse_number("123.456.789", locale = locale(grouping_mark = "."))
```

```
## [1] 123456789
```

`parse_number()` ignores non-numeric characters before and after the number.

Strings. A string is stored as bytes; `charToRaw("Hadley")` shows them in hexadecimal. The mapping from bytes to characters is the **encoding**. Modern data are almost always UTF-8, but older systems produce other encodings, e.g. Latin-1 for Western European text or EUC-KR for Korean, which print as garbage unless declared:

```
x2 <- "\xbe\xc8\xb3\xe7\xc7\xcf\xbc\xbc\xbf\xe4"     # bytes of a Korean greeting in EUC
    -KR
parse_character(x2, locale = locale(encoding = "EUC-KR"))
guess_encoding(charToRaw(x2))                        # when the encoding is unknown
```

Factors represent categorical variables with a known set of values; `parse_factor(x, levels = fruit)` warns about values not in `levels`.

Dates and times. Computers store a *date* as the number of days since 1970-01-01, a *date-time* as seconds since midnight 1970-01-01 (UTC), and a *time* as seconds since midnight.

- `parse_datetime()` expects ISO 8601, which orders components from biggest to smallest (year,

month, day, hour, minute, second); a missing time is set to midnight.

- `parse_date()` expects year, `-` or `/`, month, `-` or `/`, day.
- `parse_time()` expects hour, `:`, minutes, optionally `:` seconds and am/pm.

```
parse_datetime("2010-10-01T2010")
```

```
## [1] "2010-10-01 20:10:00 UTC"
```

```
parse_date("2010-10-01")
```

```
## [1] "2010-10-01"
```

```
parse_time("01:10 am")
```

```
## 01:10:00
```

```
parse_date("01/02/15", "%d/%m/%y")
```

```
## [1] "2015-02-01"
```

Other formats are described with codes such as `%Y` (4-digit year), `%y` (2-digit), `%m`, `%b` (abbreviated month name), `%d`, `%H`, `%M`, `%S`.

5.2.2 Parsing a file

readr guesses each column's type from the first 1000 rows, trying in order: logical (only `T`, `F`, `TRUE`, `FALSE`), integer, double, number (with grouping marks), time, date, date-time; if nothing fits, the column stays character. This fails when the first 1000 rows are a special case (e.g. all missing) or when a column has many missing values. readr's example file `challenge.csv` shows both problems:

```
challenge <- read_csv(readr_example("challenge.csv"))
problems(challenge)
```

```
## # A tibble: 0 x 5
## # i 5 variables: row <int>, col <int>, expected <chr>, actual <chr>, file <chr>
```

The column `y` has only missing values in the first 1000 rows and was guessed to be logical. Fix the column types one at a time with an explicit specification:

```
challenge <- read_csv(readr_example("challenge.csv"),
  col_types = cols(x = col_double(), y = col_date()))
tail(challenge)
```

```
## # A tibble: 6 x 2
##       x y
##   <dbl> <date>
## 1 0.805 2019-11-21
## 2 0.164 2018-03-29
## 3 0.472 2014-08-04
## 4 0.718 2015-08-16
## 5 0.270 2020-02-04
## # i 1 more row
```

Supplying `col_types` is good practice even when the guess is right: it documents the data and makes the script fail loudly if the data change. Other strategies: `guess_max = 1001` looks at more rows; reading every column as character (`cols(.default = col_character())`) and then calling `type_convert()` applies the guessing heuristic afterwards.

5.2.3 Writing files and other formats

`write_csv()` and `write_tsv()` write UTF-8 files with dates in ISO 8601, but the column types are lost: reading back re-guesses them. `write_rds()` and `read_rds()` store R's custom binary format,

which preserves types. Other formats have their own packages: `haven` (SPSS, Stata, SAS), `readxl` (`.xls`, `.xlsx`), `DBI` with a backend such as `RSQLite` (SQL databases), `jsonlite` (JSON) and `xml2` (XML).

5.3 Tidy data

Motivation. The same information can be laid out in a table in many ways, but only one layout lets the tidyverse work without fighting the data. Recognising that layout, and converting other layouts into it, is the core skill of data wrangling.

The tidyr examples `table1`–`table5` contain the same numbers (tuberculosis cases and population of three countries in 1999 and 2000) in different layouts:

```
table1
```

```
## # A tibble: 6 x 4
##   country      year  cases population
##   <chr>       <dbl>  <dbl>      <dbl>
## 1 Afghanistan  1999    745   19987071
## 2 Afghanistan  2000   2666   20595360
## 3 Brazil       1999  37737  172006362
## 4 Brazil       2000  80488  174504898
## 5 China        1999 212258 1272915272
## # i 1 more row
```

```
table2
```

```
## # A tibble: 12 x 4
##   country      year type           count
##   <chr>       <dbl> <chr>          <dbl>
## 1 Afghanistan  1999 cases            745
## 2 Afghanistan  1999 population 19987071
## 3 Afghanistan  2000 cases           2666
## 4 Afghanistan  2000 population 20595360
## 5 Brazil       1999 cases          37737
## # i 7 more rows
```

```
table3
```

```
## # A tibble: 6 x 3
##   country      year rate
##   <chr>       <dbl> <chr>
## 1 Afghanistan  1999 745/19987071
## 2 Afghanistan  2000 2666/20595360
## 3 Brazil       1999 37737/172006362
## 4 Brazil       2000 80488/174504898
## 5 China        1999 212258/1272915272
## # i 1 more row
```

Definition (Tidy data)

A data set is **tidy** if

1. each variable has its own column;
2. each observation has its own row;
3. each value has its own cell.

Only `table1` is tidy. `table2` spreads one observation over two rows; `table3` puts two variables in one cell; `table4a` (cases) and `table4b` (population) use the values of `year` as column names.

Why tidy? A consistent layout means one set of tools works everywhere, and storing variables in columns fits R's vectorised nature: `mutate()` and summary functions operate on columns.

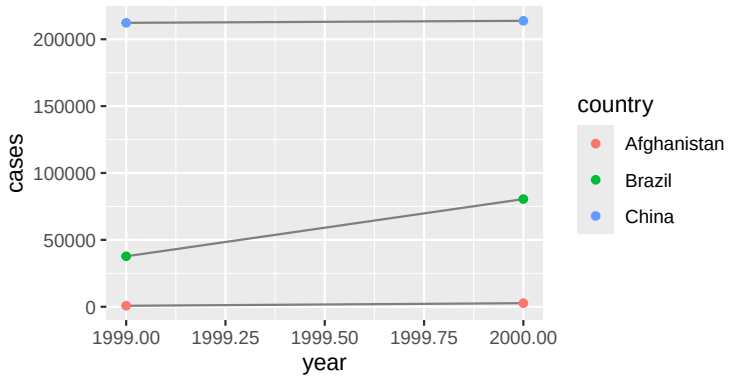
```
table1 %>% mutate(rate = cases / population * 10000)    # rate per 10,000
```

```
## # A tibble: 6 x 5
##   country      year  cases population  rate
##   <chr>       <dbl>  <dbl>      <dbl> <dbl>
## 1 Afghanistan  1999    745   19987071 0.373
## 2 Afghanistan  2000   2666   20595360 1.29
## 3 Brazil       1999  37737  172006362 2.19
## 4 Brazil       2000  80488  174504898 4.61
## 5 China        1999 212258 1272915272 1.67
## # i 1 more row
```

```
table1 %>% count(year, wt = cases)                    # cases per year
```

```
## # A tibble: 2 x 2
##    year      n
##   <dbl>  <dbl>
## 1  1999 250740
## 2  2000 296920
```

```
ggplot(table1, aes(year, cases)) +
  geom_line(aes(group = country), colour = "grey50") +
  geom_point(aes(colour = country))
```



5.4 Gathering and spreading

Motivation. Most real data are untidy in one of two ways: one variable is spread over several columns, or one observation is spread over several rows. Gathering and spreading are the two inverse operations that fix these.

Gathering. In `table4a` the column names `1999`, `2000` are values of the variable `year`, and each row holds two observations. `gather()` collects these columns into a pair of variables: the **key** (the variable whose values were column names) and the **value** (the variable whose values were in the cells).

```
table4a
```

```
## # A tibble: 3 x 3
##   country     `1999` `2000`
##   <chr>        <dbl>  <dbl>
## 1 Afghanistan    745   2666
## 2 Brazil       37737  80488
## 3 China       212258 213766
```

```
tidy4a <- table4a %>% gather(`1999`, `2000`, key = "year", value = "cases")
```

```
tidy4b <- table4b %>% gather(`1999`, `2000`, key = "year", value = "population")
left_join(tidy4a, tidy4b)
```

```
## # A tibble: 6 x 4
##   country     year   cases population
##   <chr>       <chr>  <dbl>      <dbl>
## 1 Afghanistan 1999     745   19987071
## 2 Brazil      1999   37737  172006362
## 3 China       1999  212258 1272915272
## 4 Afghanistan 2000    2666   20595360
## 5 Brazil      2000   80488  174504898
## # i 1 more row
```

The two tidied tables are combined by `left_join()` (Lecture 6). Gathering makes tables longer.

Spreading is the inverse. In `table2` an observation (country, year) is spread over two rows; `spread()` turns the values of the key column `type` into column names, filled from `count`. Spreading makes tables wider.

```
spread(table2, key = type, value = count)
```

```
## # A tibble: 6 x 4
##   country      year  cases population
##   <chr>       <dbl>  <dbl>      <dbl>
## 1 Afghanistan  1999    745   19987071
## 2 Afghanistan  2000   2666   20595360
## 3 Brazil       1999  37737  172006362
## 4 Brazil       2000  80488  174504898
## 5 China        1999 212258 1272915272
## # i 1 more row
```

pivot_longer() and pivot_wider() [Supplement]

In current tidyr, `gather()` and `spread()` are superseded by `pivot_longer()` and `pivot_wider()`, which do the same with clearer argument names; the old functions still work. The calls above become

```
table4a %>% pivot_longer(c(`1999`, `2000`), names_to = "year", values_to = "cases")
table2 %>% pivot_wider(names_from = type, values_from = count)
```

`pivot_longer()` also has `names_transform` and `values_drop_na` arguments, so the type of `year` and missing cells can be handled in the same call.

5.5 Separating and uniting

`table3` stores two variables, cases and population, in one column `rate`. `separate()` splits a column at a separator (by default any non-alphanumeric character):

```
table3 %>% separate(rate, into = c("cases", "population"), convert = TRUE)
```

```
## # A tibble: 6 x 4
##   country      year  cases population
##   <chr>       <dbl>  <int>      <int>
## 1 Afghanistan  1999    745   19987071
## 2 Afghanistan  2000   2666   20595360
## 3 Brazil       1999  37737  172006362
## 4 Brazil       2000  80488  174504898
## 5 China        1999 212258 1272915272
## # i 1 more row
```

Without `convert = TRUE` the new columns stay character, since they were split from a string. If `sep` is a number, it is a position: `separate(year, into = c("century", "year"), sep = 2)`

splits after the second character.

`unite()` is the inverse: `table5` has the year split into century and year, and

```
table5 %>% unite(new, century, year, sep = "")
```

```
## # A tibble: 6 x 3
##   country     new   rate
##   <chr>       <chr> <chr>
## 1 Afghanistan 1999  745/19987071
## 2 Afghanistan 2000  2666/20595360
## 3 Brazil      1999  37737/172006362
## 4 Brazil      2000  80488/174504898
## 5 China       1999  212258/1272915272
## # i 1 more row
```

pastes them back together (the default separator is _).

Lecture 6. Relational Data

6.1 Relational data

Motivation. Real data rarely live in a single table. Flights refer to planes, airports and weather stored elsewhere, and questions such as "are delays worse in bad weather?" need columns from several tables at once. Joins combine tables through the variables they share.

Collections of multiple related tables are called **relational data**; the relations are always defined between *pairs* of tables. Besides `flights`, `nycflights13` contains

- `airlines`: full carrier names from their abbreviated codes;
- `airports`: information about each airport, identified by its `faa` code;
- `planes`: information about each plane, identified by `tailnum`;
- `weather`: the weather at each NYC airport for each hour.

`flights` connects to `planes` via `tailnum`, to `airlines` via `carrier`, to `airports` via `origin` and `dest`, and to `weather` via `origin` (the location) and `year`, `month`, `day`, `hour` (the time).

6.1.1 Keys

A **key** is a variable (or set of variables) that uniquely identifies an observation.

- A **primary key** identifies an observation in its own table, e.g. `planes$tailnum`.
- A **foreign key** identifies an observation in another table, e.g. `flights$tailnum`, which points to a row of `planes`.

It is worth verifying that a supposed primary key really is unique:

```
planes %>% count(tailnum) %>% filter(n > 1)
```

```
## # A tibble: 0 x 2
## # i 2 variables: tailnum <chr>, n <int>
```

```
weather %>% count(year, month, day, hour, origin) %>% filter(n > 1)
```

```
## # A tibble: 3 x 6
##    year month   day  hour origin     n
##   <int> <int> <int> <int> <chr>  <int>
## 1  2013    11     3     1 EWR        2
## 2  2013    11     3     1 JFK        2
## 3  2013    11     3     1 LGA        2
```

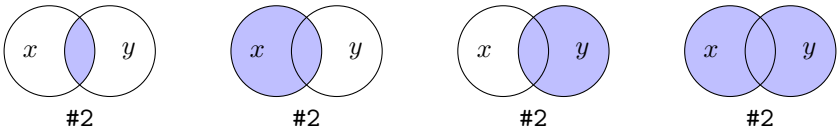
The second check finds a few duplicated hours (daylight-saving changes), so `weather` is not perfectly keyed [Supplement]. A primary key and the matching foreign key form a **relation**, typically one-to-many (each flight has one plane, each plane many flights).

6.2 Mutating joins

Motivation. A mutating join adds columns from one table to another, matching rows by key, much as mutate() adds computed columns.

We use two small tables to see exactly what each join does.

```
x <- tribble(~key, ~val_x,  1, "x1",  2, "x2",  3, "x3")
y <- tribble(~key, ~val_y,  1, "y1",  2, "y2",  4, "y3")
```



Inner join: keep pairs of rows whose keys are equal; unmatched rows are dropped (dangerous, since observations disappear silently).

```
x %>% inner_join(y, by = "key")
```

```
## # A tibble: 2 x 3
##     key val_x val_y
##   <dbl> <chr> <chr>
## 1     1 x1    y1
## 2     2 x2    y2
```

Outer joins keep observations that appear in at least one table, filling missing columns with `NA`:

- `left_join(x, y)` keeps all rows of `x`;
- `right_join(x, y)` keeps all rows of `y`;
- `full_join(x, y)` keeps all rows of both.

```
left_join(x, y, by = "key")
```

```
## # A tibble: 3 x 3
##     key val_x val_y
##   <dbl> <chr> <chr>
## 1     1 x1    y1
## 2     2 x2    y2
## 3     3 x3    <NA>
```

```
full_join(x, y, by = "key")
```

```
## # A tibble: 4 x 3
##     key val_x val_y
##   <dbl> <chr> <chr>
## 1     1 x1    y1
## 2     2 x2    y2
## 3     3 x3    <NA>
## 4     4 <NA>  y3
```

The left join is the most common: use it to look up extra information without losing any original observation.

6.2.1 Duplicate keys

If only one table has duplicate keys (a one-to-many relation), each row of the other table is copied to every match:

```
x <- tribble(~key, ~val_x,  1, "x1",  2, "x2",  2, "x3",  1, "x4")
y <- tribble(~key, ~val_y,  1, "y1",  2, "y2")
left_join(x, y, by = "key")
```

```
## # A tibble: 4 x 3
##     key val_x val_y
##   <dbl> <chr> <chr>
## 1     1 x1    y1
```

```
## 2     2 x2    y2
## 3     2 x3    y2
## 4     1 x4    y1
```

If both tables have duplicate keys, the result is every combination of matching rows (a Cartesian product within each key), which is usually an error; recent dplyr warns about an unexpected many-to-many relationship.

```
x <- tribble(~key, ~val_x,  1, "x1",  2, "x2",  2, "x3",  3, "x4")
y <- tribble(~key, ~val_y,  1, "y1",  2, "y2",  2, "y3",  3, "y4")
left_join(x, y, by = "key")
```

```
## Warning in left_join(x, y, by = "key"): Detected an unexpected many-to-many relationship
    between `x` and `y`.
## i Row 2 of `x` matches multiple rows in `y`.
## i Row 2 of `y` matches multiple rows in `x`.
## i If a many-to-many relationship is expected, set `relationship = "many-to-many"` to
##   silence this warning.
```

```
## # A tibble: 6 x 3
##     key val_x val_y
##   <dbl> <chr> <chr>
## 1     1 x1    y1
## 2     2 x2    y2
## 3     2 x2    y3
## 4     2 x3    y2
## 5     2 x3    y3
## # i 1 more row
```

6.2.2 Defining the key columns

On the real data, start from a narrower table so the added columns are visible:

```
flights2 <- flights %>% select(year:day, hour, origin, dest, tailnum, carrier)
```

- `by = NULL` (default, a **natural join**) uses all variables that appear in both tables, and reports which:

```
flights2 %>% left_join(weather)
```

```
## # A tibble: 336,776 x 18
##    year month   day  hour origin dest  tailnum carrier  temp  dewp humid wind_dir
##   <int> <int> <int> <dbl> <chr>  <chr> <chr>   <chr>   <dbl> <dbl> <dbl>    <dbl>
## 1  2013     1     1     5 EWR    IAH   N14228  UA       39.0  28.0  64.4      260
## 2  2013     1     1     5 LGA    IAH   N24211  UA       39.9  25.0  54.8      250
## 3  2013     1     1     5 JFK    MIA   N619AA  AA       39.0  27.0  61.6      260
## 4  2013     1     1     5 JFK    BQN   N804JB  B6       39.0  27.0  61.6      260
## 5  2013     1     1     6 LGA    ATL   N668DN  DL       39.9  25.0  54.8      260
## # i 336,771 more rows
## # i 6 more variables: wind_speed <dbl>, wind_gust <dbl>, precip <dbl>, pressure <dbl>,
## #   visib <dbl>, time_hour <dttm>
```

- `by = "tailnum"` uses only that common variable. `flights` and `planes` both have a `year` column with different meanings (flight year, manufacturing year), so a natural join would be wrong; with `by = "tailnum"` the two years are kept and disambiguated as `year.x`, `year.y`.
- `by = c("dest" = "faa")` matches a variable with different names in the two tables: `dest` in `flights2` and `faa` in `airports`. Joining on `c("origin" = "faa")` instead would add the origin airport's information.

```
flights2 %>% left_join(airports, c("dest" = "faa")) %>% select(dest, name, lat, lon)
```

```
## # A tibble: 336,776 x 4
##   dest  name                              lat   lon
##   <chr> <chr>                           <dbl> <dbl>
## 1 IAH   George Bush Intercontinental     30.0 -95.3
## 2 IAH   George Bush Intercontinental     30.0 -95.3
## 3 MIA   Miami Intl                       25.8 -80.3
## 4 BQN   <NA>                            NA    NA
## 5 ATL   Hartsfield Jackson Atlanta Intl  33.6 -84.4
## # i 336,771 more rows
```

6.3 Filtering joins

Motivation. Sometimes we do not want any new columns, only to keep or drop rows according to whether they have a match in another table.

Filtering joins affect rows, not columns. To restrict flights to the ten most popular destinations:

```
top_dest <- flights %>% count(dest, sort = TRUE) %>% head(10)
```

- `semi_join(x, y)` keeps the rows of `x` that *have* a match in `y`. It works for any number of key variables, where `filter(dest %in% top_dest$dest)` works only for one.
- `anti_join(x, y)` keeps the rows of `x` that *do not* have a match. It is the main tool for diagnosing join mismatches.

```
semi_join(flights, top_dest, by = "dest") %>% nrow()
```

```
## [1] 141145
```

```
flights %>%
  anti_join(planes, by = "tailnum") %>%
  count(tailnum, sort = TRUE)
```

```
## # A tibble: 722 x 2
##   tailnum     n
##   <chr>   <int>
## 1 <NA>     2512
## 2 N725MQ    575
## 3 N722MQ    513
## 4 N723MQ    507
## 5 N713MQ    483
## # i 717 more rows
```

Many flights have no plane metadata; the missing tail numbers mostly belong to two carriers (AA and MQ), which report fleet numbers rather than tail numbers [Supplement].

Checking a join [Supplement]

Before joining: identify the primary key from an understanding of the data (not by searching for a unique combination); check that no key value is missing; and check with `anti_join()` that every foreign key matches. Compare the number of rows before and after a left join: if it grows, the key was not unique.

6.4 Set operations

These treat every row as a whole, comparing all variables:

```
df1 <- tribble(~x, ~y,  1, 1,  2, 1)
df2 <- tribble(~x, ~y,  1, 1,  1, 2)
intersect(df1, df2)   # rows in both
```

```
## # A tibble: 1 x 2
##       x     y
##   <dbl> <dbl>
## 1     1     1
```

```
union(df1, df2)        # unique rows in either
```

```
## # A tibble: 3 x 2
##       x     y
##   <dbl> <dbl>
## 1     1     1
## 2     2     1
## 3     1     2
```

```
setdiff(df1, df2)     # rows in df1 but not in df2
```

```
## # A tibble: 1 x 2
##       x     y
##   <dbl> <dbl>
## 1     2     1
```

```
setdiff(df2, df1)
```

```
## # A tibble: 1 x 2
##       x     y
##   <dbl> <dbl>
## 1     1     2
```

Lecture 7. Strings

7.1 String basics

Motivation. Much real data arrives as text: names, addresses, free-form survey answers, codes that pack several pieces of information into one field. Before such data can be analysed, the information has to be found and pulled out of the strings, and stringr with regular expressions is the tool for that.

`stringr` provides consistent functions for strings, all starting with `str_`. It is part of the core tidyverse but can be loaded alone with `library(stringr)`.

Strings are written with double or single quotes; use single quotes when the string contains double quotes:

```
string1 <- "This is a string"
string2 <- 'If I want to include a "quote" inside a string, I use single quotes'
```

If you forget the closing quote, the console shows the continuation prompt `+` and waits; press Esc and try again.

Escapes. The backslash `\` escapes a special character: `"\""` is a double quote and `"\\"` a literal backslash. The printed representation of a string shows the escapes; `writeLines()` shows the raw contents:

```
x <- c("\"", "\\")
x
```

```
## [1] "\"" "\\"
```

```
writeLines(x)
```

```
## "
## \
```

Other special characters are `"\n"` (newline) and `"\t"` (tab); `?"'"` lists them all. Any Unicode character can be written as `"\u"` followed by its code, e.g. `"\uc804\uc0b0\ud1b5\uacc4"` is the course name in Hangul. Several strings are stored in a character vector, `c("one", "two", "three")`.

7.1.1 Length, combining, subsetting

```
str_length(c("a", "R for data science", NA))
```

```
## [1]  1 18 NA
```

```
str_c("x", "y", sep = ", ")
```

```
## [1] "x, y"
```

```
str_c("prefix-", c("a", "b", "c"), "-suffix")     # vectorised, recycles
```

```
## [1] "prefix-a-suffix" "prefix-b-suffix" "prefix-c-suffix"
```

```
str_c(c("x", "y", "z"), collapse = ", ")          # collapse a vector into one string
```

```
## [1] "x, y, z"
```

`str_c()` drops objects of length 0, which is convenient with `if`: in `str_c("Good ", time_of_day, " ", name, if (birthday) " and HAPPY BIRTHDAY", ".")` the clause disappears when `birthday` is `FALSE`. Missing values are contagious; `str_replace_na()` prints them as `"NA"`.

`str_sub(x, start, end)` extracts characters by position (inclusive); negative positions count from the end, and positions out of range are not an error:

```
x <- c("Apple", "Banana", "Pear")
str_sub(x, 1, 3)
```

```
## [1] "App" "Ban" "Pea"
```

```
str_sub(x, -3, -1)
```

```
## [1] "ple" "ana" "ear"
```

```
str_sub("a", 1, 5)
```

```
## [1] "a"
```

```
str_sub(x, 1, 1) <- str_to_lower(str_sub(x, 1, 1))   # assignment form
x
```

```
## [1] "apple"  "banana" "pear"
```

Positions count characters, not bytes, so `str_sub()` works for Hangul as well.

7.2 Regular expressions

Motivation. Fixed strings are too rigid for real text: we want "any digit", "a word at the start", "a repeated letter". Regular expressions are a small language for describing such patterns; they are terse, but a handful of rules covers most needs.

`str_view()` shows the matches of a pattern, marking each with `< >` (older code uses `str_view_all()`, now deprecated).

7.2.1 Basic matches

The simplest pattern matches an exact string; `.` matches any character except a newline:

```
x <- c("apple", "banana", "pear")
str_view(x, "an")
```

```
## [2] | b<an><an>a
```

```
str_view(x, ".a.")
```

```
## [2] | <ban>ana
## [3] | p<ear>
```

To match a literal `.` the regular expression needs the escape `\.`. But regular expressions are written as strings, and in a string the backslash itself must be escaped, so the R string is `"\\."`. Likewise a literal backslash needs the regular expression `\\`, i.e. the string `"\\\\"`:

```
str_view(c("abc", "a.c", "bef"), "a\\.c")
```

```
## [2] | <a.c>
```

```
writeLines("a\\b")
```

```
## a\b
```

```
str_view("a\\b", "\\\\")
```

```
## [1] | a<\>b
```

7.2.2 Anchors and character classes

`^` matches the start of the string and `$` the end ("if you begin with power (`^`), you end up with money (`$`)"). Both together force a complete match:

```
x <- c("apple pie", "apple", "apple cake", "pine apple")
```

```
str_view(x, "^apple$")
```
```
## [2] | <apple>
```

Special patterns: `\d` any digit, `\s` any whitespace, `[abc]` one of a, b, c, `[^abc]` anything except a, b, c. In a string these are written `"\\d"` and so on. Alternation `|` picks one of several alternatives, with parentheses to delimit them:

```
str_view(c("grey", "gray"), "gr(e|a)y")
```
```
## [1] | <grey>
## [2] | <gray>
```

7.2.3 Repetition

`?` means 0 or 1, `+` 1 or more, `*` 0 or more; `{n}` exactly n, `{n,}` n or more, `{,m}` at most m, `{n,m}` between n and m. Repetition is **greedy** by default (it matches the longest possible string); a `?` after the repetition makes it **lazy** (shortest match).

```
x <- "1888 is the longest year in Roman numerals: MDCCCLXXXVIII"
str_view(x, "CC+")
```
```
## [1] | 1888 is the longest year in Roman numerals: MD<CCC>LXXXVIII
```

```
str_view(x, "C[LX]+")
```
```
## [1] | 1888 is the longest year in Roman numerals: MDCC<CLXXX>VIII
```

```
str_view(x, "C{2,3}")
```
```
## [1] | 1888 is the longest year in Roman numerals: MD<CCC>LXXXVIII
```

```
str_view(x, "C{2,3}?")
```
```
## [1] | 1888 is the longest year in Roman numerals: MD<CC>CLXXXVIII
```

7.2.4 Grouping and backreferences

Parentheses also define **groups**, which can be referred to later in the pattern as `\1`, `\2`, ... Fruits containing a repeated pair of letters:

```
str_subset(fruit, "(..)\\1")
```
```
## [1] "banana"      "coconut"     "cucumber"    "jujube"      "papaya"      "salal berry"
```

7.3 Tools

Motivation. With patterns in hand we can do the real tasks: test which strings match, pull out the matching pieces, replace them, or split strings apart. Each task is one stringr function.

7.3.1 Detecting matches

`str_detect()` returns a logical vector, so counts and proportions follow from `sum()` and `mean()`. `words` is a vector of 980 common English words.

```
str_detect(c("apple", "banana", "pear"), "e")
```
```
## [1]  TRUE FALSE  TRUE
```

```
sum(str_detect(words, "^t"))          # how many words start with t?
```

```
## [1] 65
```

```
mean(str_detect(words, "[aeiou]$"))   # proportion ending with a vowel
```

```
## [1] 0.2765306
```

```
str_subset(words, "x$")               # same as words[str_detect(words, "x$")]
```

```
## [1] "box" "sex" "six" "tax"
```

Inside a data frame, use `filter(str_detect(word, "x$"))`. `str_count()` counts matches per string, e.g. the number of vowels:

```
tibble(word = words) %>%
  mutate(vowels = str_count(word, "[aeiou]"),
         consonants = str_count(word, "[^aeiou]"))
```

```
## # A tibble: 980 x 3
##   word      vowels consonants
##   <chr>      <int>      <int>
## 1 a              1          0
## 2 able           2          2
## 3 about          3          2
## 4 absolute       4          4
## 5 accept         2          4
## # i 975 more rows
```

Complex conditions are often easier with several `str_detect()` calls combined by logical operators than with a single regular expression: words without vowels are `!str_detect(words, "[aeiou]")`.

7.3.2 Extracting matches

`str_extract()` returns the first match of each string (`NA` if none), `str_extract_all()` all matches as a list (or a matrix with `simplify = TRUE`). The `sentences` data contain 720 short sentences; to find those mentioning a colour:

```
colours <- c("red", "orange", "yellow", "green", "blue", "purple")
colour_match <- str_c(colours, collapse = "|")
has_colour <- str_subset(sentences, colour_match)
head(str_extract(has_colour, colour_match))
```

```
## [1] "blue" "blue" "red"  "red"  "red"  "blue"
```

(This simple pattern also matches "red" inside "flickered"; adding word boundaries `\b` around the alternatives fixes that [Supplement].)

7.3.3 Grouped matches

`str_match()` returns the full match and each group as columns of a matrix. A rough pattern for a noun phrase, "a" or "the" followed by a word:

```
noun <- "(a|the) ([^ ]+)"
sentences %>% str_subset(noun) %>% head(4) %>% str_match(noun)
```

```
##      [,1]         [,2]  [,3]
## [1,] "the smooth" "the" "smooth"
## [2,] "the sheet"  "the" "sheet"
## [3,] "the depth"  "the" "depth"
## [4,] "a chicken"  "a"   "chicken"
```

For a tibble, `tidyr::extract()` does the same and names the groups as new columns:

```
tibble(sentence = sentences) %>%
  extract(sentence, c("article", "noun"), "(a|the) ([^ ]+)", remove = FALSE)
```

```
## # A tibble: 720 x 3
##   sentence                                    article noun
##   <chr>                                       <chr>   <chr>
## 1 The birch canoe slid on the smooth planks.  the     smooth
## 2 Glue the sheet to the dark blue background. the     sheet
## 3 It's easy to tell the depth of a well.      the     depth
## 4 These days a chicken leg is a rare dish.    a       chicken
## 5 Rice is often served in round bowls.        <NA>    <NA>
## # i 715 more rows
```

7.3.4 Replacing and splitting

`str_replace()` replaces the first match and `str_replace_all()` all matches; a named vector performs several replacements, and backreferences reorder groups:

```
x <- c("1 house", "2 cars", "3 people")
str_replace_all(x, c("1" = "one", "2" = "two", "3" = "three"))
```

```
## [1] "one house"    "two cars"     "three people"
```

```
sentences %>%
  str_replace("([^ ]+) ([^ ]+) ([^ ]+)", "\\1 \\3 \\2") %>%   # swap 2nd and 3rd words
  head(3)
```

```
## [1] "The canoe birch slid on the smooth planks."
## [2] "Glue sheet the to the dark blue background."
## [3] "It's to easy tell the depth of a well."
```

`str_split()` splits strings into pieces (a list, since each string can give a different number of pieces); `boundary("word")` splits on word boundaries rather than a single character:

```
"a|b|c|d" %>% str_split("\\|")
```

```
## [[1]]
## [1] "a" "b" "c" "d"
```

```
x <- "This is a sentence.  This is another sentence."
str_split(x, boundary("word"))[[1]]
```

```
## [1] "This"     "is"       "a"        "sentence" "This"     "is"       "another"
## [8] "sentence"
```

`str_locate()` gives the start and end positions of matches, which can be passed to `str_sub()`.

Lecture 8. Factors, Dates and Times

8.1 Factors

Motivation. Categorical variables need two things that plain strings cannot give: a fixed set of allowed values, so that typos are caught, and an order that is not alphabetical, so that months sort as months and plots show categories in a meaningful sequence. Factors provide both, and forcats makes them easy to manipulate.

8.1.1 Creating factors

Recording months as strings has two problems: only twelve values are possible but nothing protects against typos, and they sort alphabetically.

```
x1 <- c("Dec", "Apr", "Jan", "Mar")
x2 <- c("Dec", "Apr", "Jam", "Mar")
sort(x1)
```

```
## [1] "Apr" "Dec" "Jan" "Mar"
```

```
month_levels <- c("Jan", "Feb", "Mar", "Apr", "May", "Jun",
                  "Jul", "Aug", "Sep", "Oct", "Nov", "Dec")
y1 <- factor(x1, levels = month_levels)
sort(y1)
```

```
## [1] Jan Mar Apr Dec
## Levels: Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
```

```
y2 <- factor(x2, levels = month_levels)
y2
```

```
## [1] Dec  Apr  <NA> Mar
## Levels: Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
```

A factor is created from a vector and its valid **levels**; values not in the levels silently become `NA` (`readr::parse_factor()` warns instead). Without `levels`, the levels are the sorted unique values; `factor(x1, levels = unique(x1))` keeps the order of first appearance. `levels(y1)` lists the levels.

8.1.2 The General Social Survey

`forcats::gss_cat` is a sample from the General Social Survey, a long-running US survey, with many factor columns (marital status, race, income, party, religion). Factor levels can be seen with `count()` or a bar chart; `scale_x_discrete(drop = FALSE)` shows levels that have no observations.

```
gss_cat %>% count(race)
```

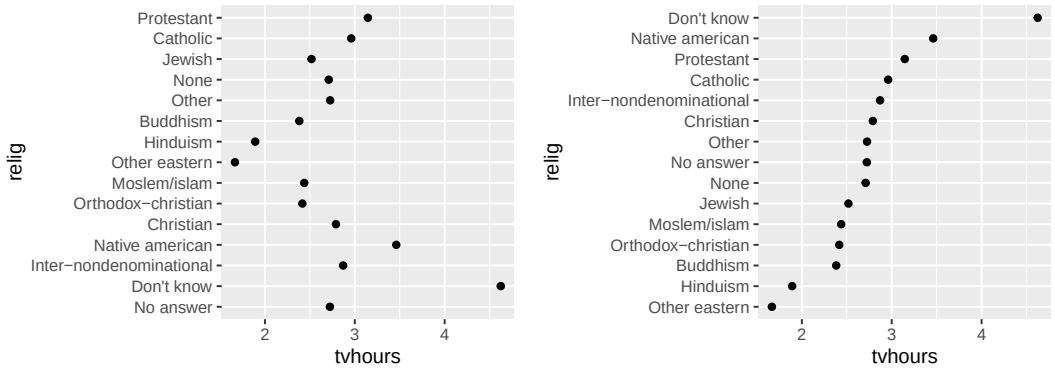
```
## # A tibble: 3 x 2
##   race      n
##   <fct> <int>
## 1 Other  1959
## 2 Black  3129
## 3 White 16395
```

The two most common operations on factors are changing the *order* of the levels and changing their *values*.

8.1.3 Modifying factor order

Average daily TV hours by religion is hard to read with the default level order. `fct_reorder(f, x)` reorders the levels of `f` by the median of `x` (or another function given as `.fun`):

```
relig_summary <- gss_cat %>%
  group_by(relig) %>%
  summarise(age = mean(age, na.rm = TRUE),
            tvhours = mean(tvhours, na.rm = TRUE), n = n())
ggplot(relig_summary, aes(tvhours, relig)) + geom_point()
relig_summary %>%
  mutate(relig = fct_reorder(relig, tvhours)) %>%
  ggplot(aes(tvhours, relig)) + geom_point()
```



Reordering inside `mutate()` rather than inside `aes()` keeps the plotting code simple. Do not reorder factors whose levels already have a natural order: for reported income, `fct_reorder(rincome, age)` would scramble the income brackets. Instead, `fct_relevel(rincome, "Not applicable")` only moves the named level(s) to the front.

Other reordering tools: `fct_reorder2(f, x, y)` orders by the `y` values at the largest `x`, so that line colours match the legend; `fct_infreq(f)` orders by frequency (useful for bar charts, combined with `fct_rev()` to reverse).

8.1.4 Modifying factor levels

`fct_recode()` renames levels (new name on the left) and leaves unmentioned levels unchanged; assigning several old levels to one new name combines them:

```
gss_cat %>%
  mutate(partyid = fct_recode(partyid,
    "Republican, strong"    = "Strong republican",
    "Republican, weak"      = "Not str republican",
    "Independent, near rep" = "Ind,near rep",
    "Independent, near dem" = "Ind,near dem",
    "Democrat, weak"        = "Not str democrat",
    "Democrat, strong"      = "Strong democrat",
    "Other"                 = "No answer",
    "Other"                 = "Don't know",
    "Other"                 = "Other party"
  )) %>%
  count(partyid)
```

```
## # A tibble: 8 x 2
##   partyid                  n
##   <fct>                <int>
## 1 Other                  548
```

```
## 2 Republican, strong     2314
## 3 Republican, weak       3032
## 4 Independent, near rep  1791
## 5 Independent            4119
## # i 3 more rows
```

Combining levels this way should be done with care, since groups that really differ get merged. To collapse many levels, `fct_collapse()` takes a vector of old levels for each new one:

```
gss_cat %>%
  mutate(partyid = fct_collapse(partyid,
    other = c("No answer", "Don't know", "Other party"),
    rep = c("Strong republican", "Not str republican"),
    ind = c("Ind,near rep", "Independent", "Ind,near dem"),
    dem = c("Not str democrat", "Strong democrat"))) %>%
  count(partyid)
```

```
## # A tibble: 4 x 2
##   partyid     n
##   <fct>   <int>
## 1 other     548
## 2 rep      5346
## 3 ind      8409
## 4 dem      7180
```

`fct_lump(f, n = 10)` keeps the `n` most frequent levels and lumps the rest into "Other", useful for plots and tables of variables with many rare levels.

8.2 Dates and times

Motivation. Time looks simple but is not: years are not all the same length, days in daylight-saving changes have 23 or 25 hours, and the same instant has different clock readings in different places. lubridate separates these issues into clear concepts so that time arithmetic gives the answer you expect.

`lubridate` (not loaded by `library(tidyverse)` in older versions) handles three types: a **date** (`<date>`), a **time** within a day (`<time>`, from the `hms` package), and a **date-time** (`<dttm>`, called POSIXct in base R), a date plus a time uniquely identifying an instant. Always use the simplest type that works: a date if you do not need the time, since date-times bring time-zone complications. `today()` and `now()` return the current date and date-time.

8.2.1 Creating dates and date-times

From strings. Name the order of year (`y`), month (`m`) and day (`d`) and lubridate parses the rest; add `_h`, `_hm`, `_hms` for times. They also accept unquoted numbers.

```
ymd("2017-01-31")
```

```
## [1] "2017-01-31"
```

```
mdy("January 31st, 2017")
```

```
## [1] "2017-01-31"
```

```
dmy("31-Jan-2017")
```

```
## [1] "2017-01-31"
```

```
ymd_hms("2017-01-31 20:11:59")
```

```
## [1] "2017-01-31 20:11:59 UTC"
```

```
mdy_hm("01/31/2017 08:01")
```

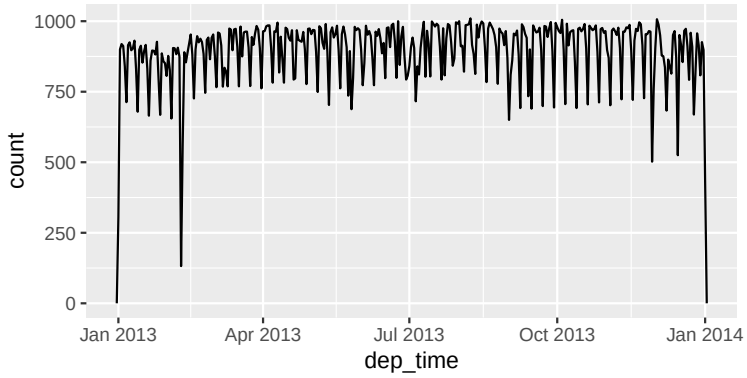
```
## [1] "2017-01-31 08:01:00 UTC"
```

From components. `make_date()` and `make_datetime()` build values from columns. In `flights`, times are stored in `HHMM` form, so we write a small helper that splits them with modular arithmetic:

```
make_datetime_100 <- function(year, month, day, time) {
  make_datetime(year, month, day, time %/% 100, time %% 100)
}
flights_dt <- flights %>%
  filter(!is.na(dep_time), !is.na(arr_time)) %>%
  mutate(
    dep_time = make_datetime_100(year, month, day, dep_time),
    arr_time = make_datetime_100(year, month, day, arr_time),
    sched_dep_time = make_datetime_100(year, month, day, sched_dep_time),
    sched_arr_time = make_datetime_100(year, month, day, sched_arr_time)
  ) %>%
  select(origin, dest, ends_with("delay"), ends_with("time"))
```

In a numeric context a date-time counts seconds (and a date counts days), so a histogram binwidth of 86400 is one day:

```
flights_dt %>%
  ggplot(aes(dep_time)) +
  geom_freqpoly(binwidth = 86400)
```

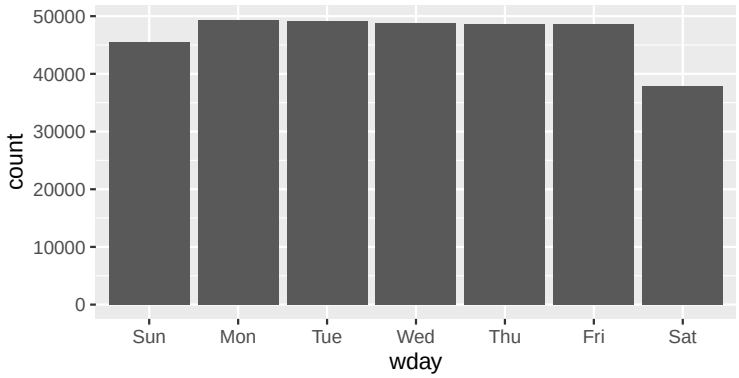


From other types. `as_datetime()` and `as_date()` switch between the two; given numbers they interpret them as an offset from the "Unix epoch" 1970-01-01.

8.2.2 Date-time components

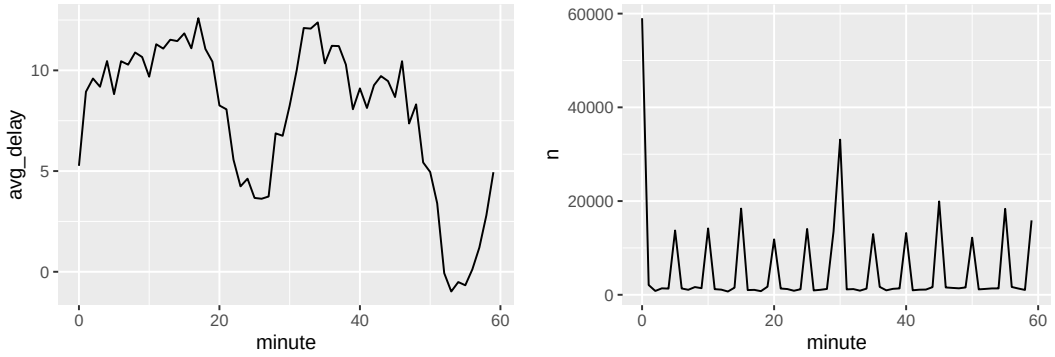
Accessors extract components: `year()`, `month()`, `mday()` (day of month), `yday()` (day of year), `wday()` (day of week), `hour()`, `minute()`, `second()`. With `label = TRUE`, `month()` and `wday()` return abbreviated names. Fewer flights leave at weekends:

```
flights_dt %>%
  mutate(wday = wday(dep_time, label = TRUE)) %>%
  ggplot(aes(x = wday)) + geom_bar()
```



The average delay by minute of *actual* departure shows that flights leaving at minutes 20–30 and 50–60 have much lower delays. By minute of *scheduled* departure there is no such pattern; the pattern comes from people preferring "nice" departure times, so early departures cluster there [Supplement].

```
flights_dt %>%
  mutate(minute = minute(dep_time)) %>%
  group_by(minute) %>%
  summarise(avg_delay = mean(arr_delay, na.rm = TRUE), n = n()) %>%
  ggplot(aes(minute, avg_delay)) + geom_line()
sched_dep <- flights_dt %>%
  mutate(minute = minute(sched_dep_time)) %>%
  group_by(minute) %>%
  summarise(avg_delay = mean(arr_delay, na.rm = TRUE), n = n())
ggplot(sched_dep, aes(minute, n)) + geom_line()
```



The right panel shows the strong preference for scheduled departures on round minutes. Look for this kind of human rounding whenever data are produced by people.

Rounding to a unit: `floor_date()`, `round_date()`, `ceiling_date()`, e.g. the number of flights per week is `count(week = floor_date(dep_time, "week"))`. **Setting** components: assign to an accessor, `year(datetime) <- 2020`, or use `update()` to change several at once; values that overflow roll over (setting `mday = 30` in February moves into March). `update(dep_time, yday = 1)` puts all flights on the same day, which shows the distribution of departures over the hours of the day.

```
datetime <- ymd_hms("2016-07-08 12:34:56")
update(datetime, year = 2020, month = 2, mday = 2, hour = 2)
```

```
## [1] "2020-02-02 02:34:56 UTC"
```

8.2.3 Time spans

Motivation. "Plus one day" can mean exactly 86 400 seconds or "the same clock time tomorrow", and these differ across a daylight-saving change. lubridate keeps them apart as durations and

periods.

Subtracting two dates gives a `difftime`, whose unit varies. lubridate offers three classes:

- **Durations** are an exact number of seconds: `as.duration(today() - ymd(19791014))`; constructors `dseconds()`, `dminutes()`, `dhours()`, `ddays()`, `dweeks()`, `dyears()`. Because they count physical seconds, adding `ddays(1)` across a daylight-saving change gives an unexpected clock time.
- **Periods** are human units: `seconds()`, `days()`, `months()`, `years()`. Adding `days(1)` keeps the clock time, and `years(1)` respects leap years. Periods fix flights that arrive after midnight: their arrival time was built on the departure date, so add `days(1)` when `arr_time < dep_time`.
- **Intervals** are a duration with a start point, `start %--% end`. Dividing an interval by a period gives the exact number of, e.g., months it contains, which a duration cannot (months have different lengths).

```
one_pm <- ymd_hms("2016-03-12 13:00:00", tz = "America/New_York")
one_pm + ddays(1)     # daylight saving started overnight: 14:00
```

```
## [1] "2016-03-13 14:00:00 EDT"
```

```
one_pm + days(1)      # same clock time: 13:00
```

```
## [1] "2016-03-13 13:00:00 EDT"
```

```
next_year <- today() + years(1)
(today() %--% next_year) / days(1)
```

```
## [1] 365
```

Rule of thumb: use a duration if you care about physical time, a period if you need human time, and an interval if you need to measure a span in human units.

8.2.4 Time zones

Time zones are named by "continent/city" (IANA names such as `"Asia/Seoul"`, `"America/New_York"`); `Sys.timezone()` gives yours and `OlsonNames()` lists all. The time zone only affects printing: the same instant printed in three zones is still the same instant, and the differences between them are 0. lubridate uses UTC (Coordinated Universal Time) by default, which has no daylight saving. To change the time zone:

- `with_tz(x, "Asia/Seoul")` keeps the instant and changes the display;
- `force_tz(x, "Asia/Seoul")` keeps the clock reading and changes the instant, for values that were labelled with the wrong time zone.

```
x <- ymd_hms("2024-06-01 12:00:00", tz = "UTC")
with_tz(x, "Asia/Seoul")
```

```
## [1] "2024-06-01 21:00:00 KST"
```

```
force_tz(x, "Asia/Seoul") - x
```

```
## Time difference of -9 hours
```

Lecture 9. Pipes

9.1 Why pipes

Motivation. This lecture starts the programming part of the course. Code is a means of communication, with the computer and with the people (including your future self) who will read it. The pipe is the first tool for writing code that reads in the same order as the analysis it performs.

The pipe `%>%` comes from the `magrittr` package; tidyverse packages load it automatically. To see why it helps, write the same sequence of actions four ways, using the nursery rhyme "Little bunny Foo Foo went hopping through the forest, scooping up the field mice and bopping them on the head".

Save each intermediate step.

```
foo_foo <- little_bunny()
foo_foo_1 <- hop(foo_foo, through = forest)
foo_foo_2 <- scoop(foo_foo_1, up = field_mice)
foo_foo_3 <- bop(foo_foo_2, on = head)
```

The code is cluttered with unimportant names, and the suffixes must be incremented carefully (it is easy to use `foo_foo_1` where `foo_foo_2` was meant). Worry about memory is usually misplaced: R shares columns between modified copies of a data frame, so copies are cheap until a column changes.

Overwrite the original.

```
foo_foo <- hop(foo_foo, through = forest)
foo_foo <- scoop(foo_foo, up = field_mice)
foo_foo <- bop(foo_foo, on = head)
```

Debugging is painful (a mistake means rerunning from the start), and the repeated name hides what changes on each line.

Function composition.

```
bop(scoop(hop(foo_foo, through = forest), up = field_mice), on = head)
```

The code must be read inside out and right to left, and each function's arguments end up far from its name (the "Dagwood sandwich" problem).

The pipe.

```
foo_foo %>%
  hop(through = forest) %>%
  scoop(up = field_mice) %>%
  bop(on = head)
```

The code reads as a sequence of verbs, "Foo Foo hops, then scoops, then bops". Behind the scenes magrittr rewrites it as a function with a placeholder `.`, roughly

```
my_pipe <- function(.) {
  . <- hop(., through = forest)
  . <- scoop(., up = field_mice)
  bop(., on = head)
}
```

Consequently the pipe does not work with functions that use the current environment (`assign()`, `get()`, `load()`) or rely on lazy evaluation in a special way (`tryCatch()`, `try()`, `suppressMessages()`).

The native pipe [Supplement]

Since R 4.1 base R has its own pipe `|>`, which works like `%>%` in simple cases: `x |> f(y)` is `f(x, y)`. It is slightly faster and needs no package, but it has no `.` placeholder in the same form (R 4.2 added `_` for named arguments) and none of the special pipes below. These notes use `%>%`, as the course does.

9.2 When not to use the pipe

Pipes are for a linear sequence of operations on one object. Use something else when

- the pipe is longer than about ten steps: create intermediate objects with meaningful names, which also makes debugging easier;
- there are several inputs or outputs, i.e. no single primary object is being transformed;
- the steps form a directed graph with complex dependencies rather than a chain.

9.3 Other tools from magrittr

The tee pipe `%T>%` returns its left-hand side instead of the right-hand side, so a step with a side effect (plotting, printing, saving) does not end the pipeline. Without it, `plot()` returns `NULL` and the next step receives nothing:

```
rnorm(100) %>%
  matrix(ncol = 2) %>%
  plot() %>%
  str()
```

```
##  NULL
```

```
rnorm(100) %>%
  matrix(ncol = 2) %T>%
  plot() %>%
  str()
```

```
##  num [1:50, 1:2] -0.387 -0.785 -1.057 -0.796 -1.756 ...
```

The exposition pipe `%$%` "explodes" the variables of a data frame so they can be referred to by name in functions without a `data` argument:

```
mtcars %$% cor(disp, mpg)
```

```
## [1] -0.8475514
```

The assignment pipe `%<>%` replaces `mtcars <- mtcars %>% transform(cyl = cyl * 2)` by `mtcars %<>% transform(cyl = cyl * 2)`. Many prefer to avoid it: assignment is a special operation that should be clearly visible at the start of a line.

Lecture 10. Functions

10.1 When to write a function

Motivation. Copy-and-paste is the most common source of bugs in analysis code: one copy gets updated and another does not. Functions remove the copies. They also give a name to an idea, which makes code easier to read and to change.

Consider rescaling each column of a data frame to $[0, 1]$:

```
df <- tibble::tibble(a = rnorm(10), b = rnorm(10), c = rnorm(10), d = rnorm(10))
df$a <- (df$a - min(df$a, na.rm = TRUE)) /
  (max(df$a, na.rm = TRUE) - min(df$a, na.rm = TRUE))
df$b <- (df$b - min(df$b, na.rm = TRUE)) /
  (max(df$b, na.rm = TRUE) - min(df$a, na.rm = TRUE))
# ... and the same for c and d
```

There is a copy-paste error in the `b` line (`min(df$a)`) that is easy to miss. A rule of thumb: write a function whenever you have copied a block of code more than twice (DRY, "do not repeat yourself").

How to write one. First analyse the code: how many inputs does it have? Here one, so rename it to a temporary general name, `x`. Then remove duplication: the code computes the range three times, and `range()` does it once:

```
rescale01 <- function(x) {
  rng <- range(x, na.rm = TRUE)
  (x - rng[1]) / (rng[2] - rng[1])
}
rescale01(c(-10, 0, 10))
```

```
## [1] 0.0 0.5 1.0
```

```
rescale01(c(1, 2, 3, NA, 5))
```

```
## [1] 0.00 0.25 0.50   NA 1.00
```

Three steps create a function: pick a **name**, list the **arguments** inside `function()`, and put the code in the **body** in braces. It is easier to start with working code and turn it into a function than the reverse. The four columns now take one line each, `df$a <- rescale01(df$a)`, and Lecture 12 removes even that repetition.

A further advantage: requirements change in one place. If some values are infinite, the range is infinite and everything becomes `NaN`; one change to the function fixes every use:

```
x <- c(1:10, Inf)
rescale01 <- function(x) {
  rng <- range(x, na.rm = TRUE, finite = TRUE)
  (x - rng[1]) / (rng[2] - rng[1])
}
rescale01(x)
```

```
##  [1] 0.0000000 0.1111111 0.2222222 0.3333333 0.4444444 0.5555556 0.6666667 0.7777778
##  [9] 0.8888889 1.0000000       Inf
```

10.2 Functions are for humans and computers

R does not care about names or comments, but readers do.

- **Names** should be short but clearly describe what the function does: verbs for functions

(`impute_missing()`, `collapse_years()`), nouns for arguments. Avoid both too short (`f()`) and undescriptive names (`my_awesome_function()`).

- Use one naming style consistently, `snake_case` or `camelCase`, never a mix like `col_mins()` and `rowMaxes()`. A common prefix groups a family of functions for autocompletion: `input_select()`, `input_checkbox()`, `input_text()`.
- Never overwrite existing functions and variables: `T <- FALSE`, `c <- 10`, `mean <- function(x) sum(x)` lead to baffling bugs.
- **Comments** explain the *why*, not the what or how, which the code itself should make clear. Lines of dashes such as `# Load data ---` break a script into readable sections (RStudio can jump between them).

10.3 Conditional execution

Motivation. A function often has to behave differently for different inputs: handle a special case, check a condition, choose between methods. if–else statements express these branches.

```
if (condition) {
  # code executed when condition is TRUE
} else {
  # code executed when condition is FALSE
}
```

```
has_name <- function(x) {
  nms <- names(x)
  if (is.null(nms)) {
    rep(FALSE, length(x))
  } else {
    !is.na(nms) & nms != ""
  }
}
has_name(c(a = 1, 2, b = 3))
```

```
## [1]  TRUE FALSE  TRUE
```

This uses the convention that a function returns the last value it computed.

10.3.1 Conditions

The condition must be a single `TRUE` or `FALSE`; a longer logical vector is an error (it used to be a warning), and `NA` is an error:

```
if (c(TRUE, FALSE)) {}
```

```
## Error in if (c(TRUE, FALSE)) {: the condition has length > 1
```

```
if (NA) {}
```

```
## Error in if (NA) {: missing value where TRUE/FALSE needed
```

Use `||` and `&&` (which look at one value and stop as soon as the answer is known) to combine conditions, never the vectorised `|` and `&` inside `if`. If you have a logical vector, summarise it with `any()` or `all()`. Be careful with `==`: it is vectorised and returns `NA` for missing values. `identical()` is strict, returning one `TRUE` or `FALSE` but also distinguishing types (`identical(0L, 0)` is `FALSE`); for real numbers use `dplyr::near()` (Lecture 3), since `sqrt(2)^2 == 2` is `FALSE`.

10.3.2 Multiple conditions

Chain `if (this) {...} else if (that) {...} else {...}`. With many alternatives, `switch()` selects code by name or position:

```
calc <- function(x, y, op) {
  switch(op,
    plus = x + y,
    minus = x - y,
    times = x * y,
    divide = x / y,
    stop("Unknown op!")
  )
}
calc(6, 3, "divide")
```

```
## [1] 2
```

`cut()` discretises continuous variables, replacing long `if` chains.

10.3.3 Code style

An opening brace is never on its own line and is always followed by a new line; a closing brace always goes on its own line, unless followed by `else`. Indent the contents by two spaces. A very short `if` can sit on one line: `x <- if (y < 20) "Too low" else "Too high"`. Putting `else` on a new line after a closing brace is a syntax error at the top level, because R considers the `if` statement complete.

10.4 Function arguments

Motivation. Arguments are the interface of a function. A good interface separates what the data are from how to compute, gives sensible defaults, and fails loudly on invalid input.

Arguments fall into two sets: those supplying the **data** to compute on, and those controlling the **details** of the computation. Data arguments come first; detail arguments come last and usually have **default values**:

```
mean_ci <- function(x, conf = 0.95) {
  se <- sd(x) / sqrt(length(x))
  alpha <- 1 - conf
  mean(x) + se * qnorm(c(alpha / 2, 1 - alpha / 2))
}
x <- runif(100)
mean_ci(x)
```

```
## [1] 0.4194814 0.5276171
```

```
mean_ci(x, conf = 0.99)
```

```
## [1] 0.4024921 0.5446064
```

The default should almost always be the most common value (an exception is `na.rm = FALSE`: missing values are important, so ignoring them should be a deliberate choice). When calling, omit the names of data arguments and name detail arguments in full: `mean(1:10, na.rm = TRUE)`, not `mean(, TRUE, x = c(1:10, NA))`. Put spaces around `=` in calls and after commas.

Common argument names: `x`, `y`, `z` (vectors), `w` (weights), `df` (data frame), `i`, `j` (indices), `n` (length or number of rows), `p` (number of columns). Follow existing conventions such as `na.rm`.

10.4.1 Checking values

Recycling rules make some mistakes silent:

```
wt_mean <- function(x, w) sum(x * w) / sum(w)
wt_mean(1:6, 1:3)
```

```
## [1] 7.666667
```

The weights were recycled to length 6 without any warning (the length 6 is a multiple of 3). Check important preconditions and throw an error with `stop()`, or use `stopifnot()`, which checks that each argument is `TRUE`:

```
wt_mean <- function(x, w, na.rm = FALSE) {
  stopifnot(is.logical(na.rm), length(na.rm) == 1)
  stopifnot(length(x) == length(w))
  if (na.rm) {
    miss <- is.na(x) | is.na(w)
    x <- x[!miss]
    w <- w[!miss]
  }
  sum(w * x) / sum(w)
}
wt_mean(1:6, 6:1, na.rm = "foo")
```

```
## Error in wt_mean(1:6, 6:1, na.rm = "foo"): is.logical(na.rm) is not TRUE
```

Do not overdo it: checks are a trade-off between time spent and robustness.

10.4.2 Dot-dot-dot

Functions such as `sum()` take any number of inputs through the special argument `...`, which captures all unmatched arguments and can be passed on to another function:

```
commas <- function(...) stringr::str_c(..., collapse = ", ")
commas(letters[1:10])
```

```
## [1] "a, b, c, d, e, f, g, h, i, j"
```

The price is that misspelled argument names are swallowed without error: `sum(x, na.mr = TRUE)` adds `TRUE` (that is, 1) to the sum.

10.5 Return values

By default a function returns the last value it evaluates. `return()` returns early, which is useful when the input is a simple special case, or to keep the long branch of an `if` un-indented:

```
complicated_function <- function(x, y, z) {
  if (length(x) == 0 || length(y) == 0) {
    return(0)
  }
  # complicated code here
}
```

Pipeable functions. There are two types: *transformations* return a modified version of their first argument; *side-effect* functions (plotting, printing, saving) should return their first argument *invisibly*, so that it is not printed but can still be piped on:

```
show_missings <- function(df) {
  n <- sum(is.na(df))
  cat("Missing values: ", n, "\n", sep = "")
```

```
  invisible(df)
}
mtcars %>%
  show_missings() %>%
  mutate(mpg = ifelse(mpg < 20, NA, mpg)) %>%
  show_missings()
```

```
## Missing values: 0
## Missing values: 18
```

10.6 Environment

Motivation. A function body can mention variables that are not its arguments. Knowing where R looks for them explains both a common source of bugs and why R functions can be so flexible.

```
f <- function(x) {
  x + y
}
y <- 100
f(10)
```

```
## [1] 110
```

```
y <- 1000
f(10)
```

```
## [1] 1010
```

In many languages `f` would be an error because `y` is not defined inside it. R uses **lexical scoping**: a name not found inside the function is looked up in the environment where the function was *defined*, here the global environment. The result therefore depends on the current value of a global variable, which is a recipe for bugs; avoid writing functions like this deliberately. The same rules let R override even `+` or `(`, which makes the language extremely flexible.

Defined, not called [Supplement]

Lexical scoping looks where the function was created, not where it is called:

```
make_adder <- function(y) function(x) x + y
add2 <- make_adder(2)
y <- 1000
add2(10)
```

```
## [1] 12
```

`add2` remembers the `y = 2` of the environment in which it was created (a *closure*); the global `y` is irrelevant.

Lecture 12. Iteration

12.1 Iteration

Motivation. Functions removed duplicated code by naming a computation; iteration removes it by repeating a computation over many inputs, such as every column of a data frame or every group of a data set. R offers two styles, explicit loops and functional programming, and both are worth knowing.

Reducing duplication has three benefits: the intent of the code is easier to see, requirements change in one place, and there are fewer bugs because each line is used in more places. Lecture 10 used functions; this lecture uses **iteration**: *imperative* for and while loops, which make iteration explicit, and *functional* tools (`purrr`), which hide the boilerplate.

12.2 For loops

To compute the median of each column of

```
df <- tibble(a = rnorm(10), b = rnorm(10), c = rnorm(10), d = rnorm(10))
```

copying `median(df$a)` four times breaks the rule of thumb. A for loop:

```
output <- vector("double", ncol(df))   # 1. output
for (i in seq_along(df)) {             # 2. sequence
  output[[i]] <- median(df[[i]])       # 3. body
}
output
```

```
## [1] -0.24576245 -0.28730721 -0.05669771  0.14426335
```

Every loop has three components.

1. **Output**: allocate enough space before the loop with `vector(type, length)`. Growing a vector with `c()` at each step is very slow, because the whole vector is copied every time.
2. **Sequence**: `i in seq_along(df)` gives a different value of `i` on each run. `seq_along()` is a safe version of `1:length()`: for a zero-length vector it gives an empty sequence, whereas `1:length(y)` gives `c(1, 0)` and runs the body on nonsense indices.
3. **Body**: the code that does the work, run once per value of `i`. Use `[[ ]]` even for atomic vectors, to make clear that one element is handled.

```
y <- vector("double", 0)
seq_along(y)
```

```
## integer(0)
```

```
1:length(y)
```

```
## [1] 1 0
```

12.2.1 Variations

Modifying an existing object. Rescaling every column of `df` (Lecture 10):

```
rescale01 <- function(x) {
  rng <- range(x, na.rm = TRUE)
```

```
  (x - rng[1]) / (rng[2] - rng[1])
}
for (i in seq_along(df)) {
  df[[i]] <- rescale01(df[[i]])
}
```

Here the output is `df` itself, so no new object is needed.

Looping patterns. Besides looping over indices, one can loop over the elements, `for (x in xs)`, which is awkward for saving output, or over names, `for (nm in names(xs))`, accessing values with `x[[nm]]`. Looping over indices is the most general: from the position one gets both the name and the value:

```
results <- vector("list", length(x))
names(results) <- names(x)
for (i in seq_along(x)) {
  name <- names(x)[[i]]
  value <- x[[i]]
}
```

Unknown output length. If each iteration produces a vector of random length, do not grow the result with `c()` (quadratic time). Store the pieces in a list and combine once at the end, with `unlist()` (or `purrr::flatten_dbl()`) for vectors, `paste(collapse = "")` for strings, and `dplyr::bind_rows()` for data frames:

```
means <- c(0, 1, 2)
out <- vector("list", length(means))
for (i in seq_along(means)) {
  n <- sample(100, 1)
  out[[i]] <- rnorm(n, means[[i]])
}
str(out)
```

```
## List of 3
##  $ : num [1:38] 0.255 -0.553 1.405 -0.795 -1.567 ...
##  $ : num [1:17] 0.91 2.92 2.3 1.75 1.56 ...
##  $ : num [1:14] 3.215 0.722 2.748 5.392 3.619 ...
```

```
str(unlist(out))
```

```
##  num [1:69] 0.255 -0.553 1.405 -0.795 -1.567 ...
```

Unknown sequence length: while loops. A `while` loop runs while a condition is `TRUE`; every for loop can be written as a while loop but not vice versa. Simulations often need it, e.g. the number of coin flips until three heads in a row:

```
flip <- function() sample(c("T", "H"), 1)
flips <- 0
nheads <- 0
while (nheads < 3) {
  if (flip() == "H") nheads <- nheads + 1 else nheads <- 0
  flips <- flips + 1
}
flips
```

```
## [1] 18
```

How many flips on average? [Supplement]

Let e_k be the expected number of further flips when the current run of heads has length k. Then $e_3 = 0$ and $e_k = 1 + \frac{1}{2}e_{k+1} + \frac{1}{2}e_0$ for $k = 0, 1, 2$. Solving gives $e_0 = 14$. A simulation agrees:

```r
flips_until_3 <- function() {
  flips <- 0; nheads <- 0
  while (nheads < 3) {
    if (flip() == "H") nheads <- nheads + 1 else nheads <- 0
    flips <- flips + 1
  }
  flips
}
mean(replicate(5000, flips_until_3()))
```

```
## [1] 13.9762
```

12.3 For loops versus functionals

Motivation. Loops written for each new summary differ only in the function they apply. Passing that function as an argument removes the remaining duplication; this idea, functions that take functions, is what purrr is built on.

Column means, medians and standard deviations each need a nearly identical loop. The only difference is the function called, so make it an argument:

```r
col_summary <- function(df, fun) {
  out <- vector("double", length(df))
  for (i in seq_along(df)) {
    out[i] <- fun(df[[i]])
  }
  out
}
col_summary(df, median)
```

```
## [1] 0.6111823 0.4011430 0.4280338 0.5813702
```

```r
col_summary(df, sd)
```

```
## [1] 0.3117016 0.2993812 0.3078818 0.2728141
```

A function that takes another function as input is a **functional**. Base R's `apply()`, `lapply()`, `tapply()` are functionals; `purrr` provides a consistent family.

12.4 The map functions

Looping over a vector, doing something to each element and saving the results is so common that purrr has a function for each output type:

function	output
`map()`	list
`map_lgl()`	logical vector
`map_int()`	integer vector
`map_dbl()`	double vector
`map_chr()`	character vector

```r
df %>% map_dbl(mean)
```

```
##         a         b         c         d
## 0.5566313 0.4287933 0.4083740 0.5836067
```

```r
df %>% map_dbl(sd)
```

```
##         a         b         c         d
## 0.3117016 0.2993812 0.3078818 0.2728141
```

Compared with `col_summary()`, map functions are written in C (fast), pass extra arguments through `...` (`map_dbl(df, mean, trim = 0.5)`), and keep names. The focus moves from the bookkeeping of the loop to the operation performed; this does not make loops "slow" (that is a myth when output is preallocated), it makes code clearer.

12.4.1 Shortcuts

To fit a linear model to each cylinder group of `mtcars`, `split()` the data into a list of data frames and map a function over it. Anonymous functions can be written in a one-sided formula, where `.` (or `.x`) is the current element:

```
models <- mtcars %>%
  split(.$cyl) %>%
  map(~lm(mpg ~ wt, data = .))
models %>%
  map(summary) %>%
  map_dbl(~.x$r.squared)
```

```
##         4         6         8
## 0.5086326 0.4645102 0.4229655
```

Extracting a named component is so common that a string suffices, and an integer selects by position:

```
models %>% map(summary) %>% map_dbl("r.squared")
```

```
##         4         6         8
## 0.5086326 0.4645102 0.4229655
```

```
x <- list(list(1, 2, 3), list(4, 5, 6), list(7, 8, 9))
x %>% map_dbl(2)
```

```
## [1] 2 5 8
```

12.4.2 Base R

`lapply()` is essentially `map()`. `sapply()` simplifies the output automatically, which is convenient interactively but dangerous in functions, because the output type depends on the data:

```
threshold <- function(x, cutoff = 0.8) x[x > cutoff]
x1 <- list(c(0.27, 0.37, 0.57, 0.91, 0.20), c(0.90, 0.94, 0.66, 0.63, 0.06),
           c(0.21, 0.18, 0.69, 0.38, 0.77))
x2 <- list(c(0.50, 0.72, 0.99, 0.38, 0.78), c(0.93, 0.21, 0.65, 0.13, 0.27),
           c(0.39, 0.01, 0.38, 0.87, 0.34))
x1 %>% sapply(threshold) %>% str()
```

```
## List of 3
##  $ : num 0.91
##  $ : num [1:2] 0.9 0.94
##  $ : num(0)
```

```
x2 %>% sapply(threshold) %>% str()
```

```
##  num [1:3] 0.99 0.93 0.87
```

The first returns a list, the second a numeric vector. `vapply()` is a safe alternative that requires the output type, `vapply(df, is.numeric, logical(1))`.

12.5 Dealing with failure

Motivation. When mapping over many inputs, one failure stops everything and loses all the results that worked. purrr's adverbs turn errors into data, so failures can be inspected after the run.

`safely()` wraps a function so that it never throws an error; it returns a list with `result` and `error`, one of which is `NULL`:

```
safe_log <- safely(log)
str(safe_log(10))
```

```
## List of 2
##  $ result: num 2.3
##  $ error : NULL
```

```
x <- list(1, 10, "a")
y <- x %>% map(safely(log)) %>% transpose()
is_ok <- y$error %>% map_lgl(is_null)
x[!is_ok]
```

```
## [[1]]
## [1] "a"
```

```
y$result[is_ok] %>% flatten_dbl()
```

```
## [1] 0.000000 2.302585
```

`transpose()` turns a list of pairs into a pair of lists. `possibly()` returns a default value on error, `x %>% map_dbl(possibly(log, NA_real_))`; `quietly()` captures printed output, messages and warnings instead of errors.

12.6 Mapping over multiple arguments

To simulate normal samples with varying means, `map(mu, rnorm, n = 5)` varies one argument. To vary both mean and standard deviation, `map2()` iterates over two vectors in parallel; `pmap()` takes a list of any number of arguments (named, to match them safely). A data frame is a list of equal-length columns, so a tibble of parameters drives `pmap()` directly:

```
mu <- list(5, 10, -3)
sigma <- list(1, 5, 10)
map2(mu, sigma, rnorm, n = 5) %>% str()
```

```
## List of 3
##  $ : num [1:5] 5.45 4.52 4.16 5.48 2.03
##  $ : num [1:5] 17.59 1.02 10.05 8.09 6.07
##  $ : num [1:5] -6.78 7.76 14.72 -6.58 -11.58
```

```
params <- tribble(
  ~mean, ~sd, ~n,
      5,   1,  1,
     10,   5,  3,
     -3,  10,  5
)
params %>% pmap(rnorm) %>% str()
```

```
## List of 3
##  $ : num 3.99
##  $ : num [1:3] 13.2 14.4 12.6
##  $ : num [1:5] -8.43 4.12 -12.89 5.54 -15.36
```

`map2()` is just a loop in disguise: it allocates a list and calls `f(x[[i]], y[[i]], ...)` for each `i`. To vary the function itself as well, combine `map2()` with `exec()`, e.g. `map2(c("runif", "rnorm",`

`"rpois"), param, function(fn, args) exec(fn, !!!args))` (the older `invoke_map()` is deprecated).

12.7 Walk, reduce and accumulate

walk() is used for side effects (printing, saving files) instead of return values; `walk()`, `walk2()`, `pwalk()` return their first argument invisibly, e.g. `pwalk(list(paths, plot = plots), ggsave, path = tempdir())` saves a list of plots to files.

reduce() combines a list into one value by applying a two-argument function repeatedly. To join a list of data frames on a common key, `dfs %>% reduce(full_join)`; to find values common to all vectors, `vs %>% reduce(intersect)`. **accumulate()** keeps the intermediate results:

```
x <- sample(10)
x
```

```
##  [1]  1  7  8  2  6 10  3  9  4  5
```

```
x %>% accumulate(`+`)
```

```
##  [1]  1  8 16 18 24 34 37 46 50 55
```

Lecture 13. Model Basics

13.1 Models and their purpose

Motivation. EDA finds patterns by eye; a model describes a pattern precisely enough to measure it, predict from it and subtract it. This lecture builds the idea of a model from scratch on simulated data, so that what lm() does is no longer a black box.

The goal of a model is a simple low-dimensional summary of a data set: to partition the data into *patterns* and *residuals*. In this course models are tools for exploration, not for inference (p-values, confidence intervals).

Hypothesis generation versus confirmation. Each observation can be used for exploration or for confirmation, not both. Exploration may use an observation as often as you like; confirmation may use it only once. As soon as an observation is used twice you have switched from confirmation to exploration. For confirmatory analysis, split the data in advance into a *training* set (about 60%) for exploration and model fitting, a *query* set (about 20%) to compare models by hand, and a *test* set (about 20%) used only once to test the final model.

A model has two parts:

1. a **family of models** expressing a precise but generic pattern, e.g. the lines $y = a_1 + a_2 x$;
2. a **fitted model**, the member of the family closest to the data, e.g. $y = 3x + 7$.

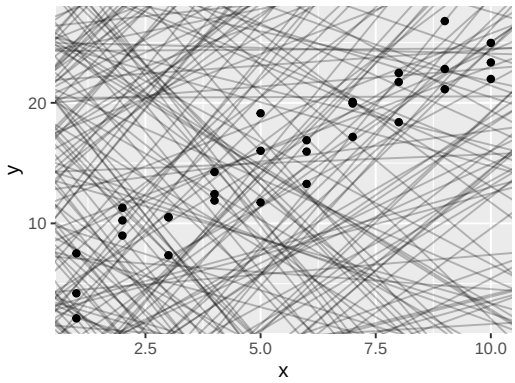
A fitted model is only the closest model from a family; it does not mean the model is "true". "All models are wrong, but some are useful" (G. E. P. Box).

13.2 A simple model

Motivation. Fitting a line seems trivial, but doing it by hand exposes the two ingredients of every fitting method: a measure of distance between model and data, and a search for the parameters that minimise it.

We use `modelr`, whose `options(na.action = na.warn)` makes models warn about dropped missing values. The simulated data `sim1` has a clear linear pattern. Generate many random lines and overlay them:

```
models <- tibble(a1 = runif(250, -20, 40), a2 = runif(250, -5, 5))
ggplot(sim1, aes(x, y)) +
  geom_abline(aes(intercept = a1, slope = a2), data = models, alpha = 1/4) +
  geom_point()
```



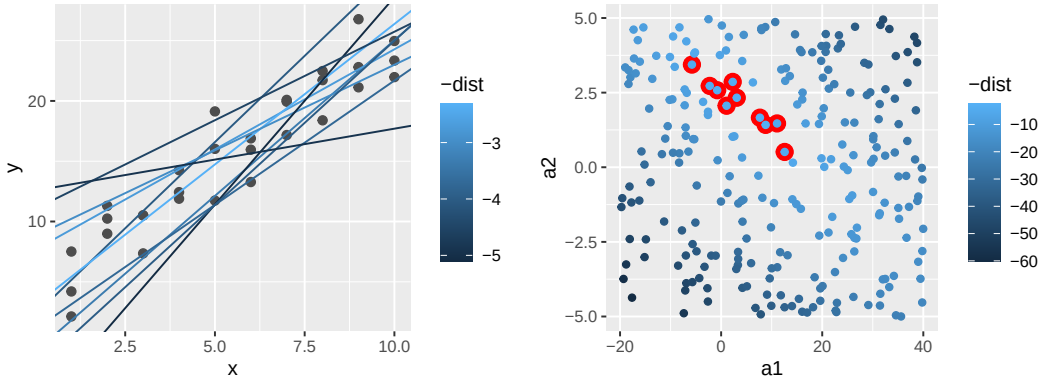
To pick the best line we need the distance between data and model. Write the model as a function of the parameters and data, and measure the distance by the root-mean-squared vertical deviation:

```
model1 <- function(a, data) a[1] + data$x * a[2]
measure_distance <- function(mod, data) {
  diff <- data$y - model1(mod, data)
  sqrt(mean(diff ^ 2))
}
measure_distance(c(7, 1.5), sim1)
```

```
## [1] 2.665212
```

Computing the distance for every random model needs a helper with two scalar arguments, since `map2_dbl()` passes one `a1` and one `a2` at a time:

```
sim1_dist <- function(a1, a2) measure_distance(c(a1, a2), sim1)
models <- models %>% mutate(dist = map2_dbl(a1, a2, sim1_dist))
ggplot(sim1, aes(x, y)) +
  geom_point(size = 2, colour = "grey30") +
  geom_abline(aes(intercept = a1, slope = a2, colour = -dist),
              data = filter(models, rank(dist) <= 10))
ggplot(models, aes(a1, a2)) +
  geom_point(data = filter(models, rank(dist) <= 10), size = 4, colour = "red") +
  geom_point(aes(colour = -dist))
```



The right panel shows each model as a point in parameter space; the ten best are highlighted. A **grid search** replaces the random points by an evenly spaced grid (`expand.grid()`) around the best region and repeats. Better still is a numerical minimiser: `optim()` uses Newton–Raphson-type steps, starting from a point and moving downhill until it cannot improve:

```
best <- optim(c(0, 0), measure_distance, data = sim1)
best$par
```

```
## [1] 4.222248 2.051204
```

For linear models there is a closed form, which `lm()` computes exactly (by QR decomposition rather than iteration):

```
sim1_mod <- lm(y ~ x, data = sim1)
coef(sim1_mod)
```

```
## (Intercept)           x
##    4.220822    2.051533
```

Why the two agree [Supplement]

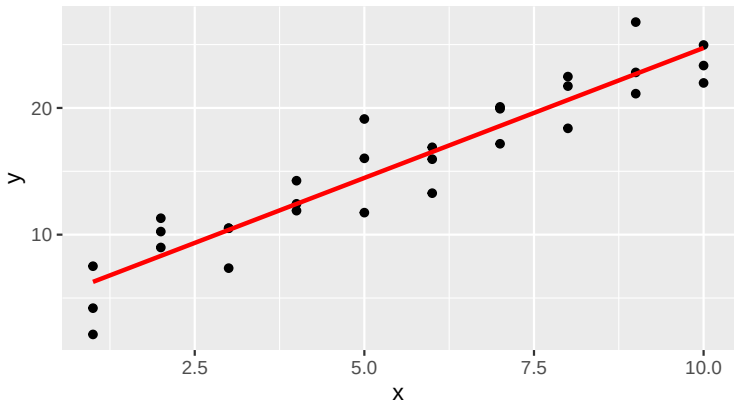
Minimising the root-mean-squared distance is equivalent to minimising the sum of squares $\sum_i (y_i - a_1 - a_2 x_i)^2$, since $\sqrt{\cdot}$ and $1/n$ are increasing. That is the least-squares problem, whose solution is $\hat{a}_2 = \sum (x_i - \bar{x})(y_i - \bar{y}) / \sum (x_i - \bar{x})^2$ and $\hat{a}_1 = \bar{y} - \hat{a}_2 \bar{x}$; `optim()` only approximates it numerically.

13.3 Visualising models

Motivation. Coefficients are hard to interpret once models grow. A more general approach looks at what a model predicts and what it leaves unexplained, and this works for every kind of model.

Predictions. Build an evenly spaced grid covering the region of the data with `data_grid()`, then add the model's predictions with `add_predictions()`:

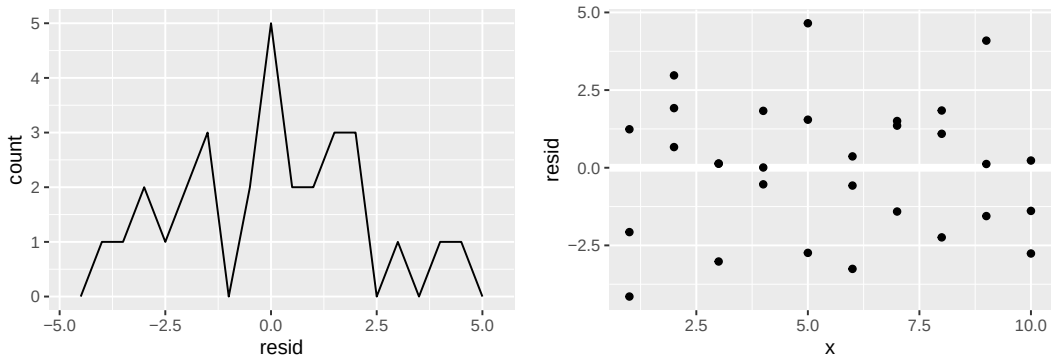
```
grid <- sim1 %>% data_grid(x) %>% add_predictions(sim1_mod)
ggplot(sim1, aes(x)) +
  geom_point(aes(y = y)) +
  geom_line(aes(y = pred), data = grid, colour = "red", linewidth = 1)
```



This works for any model, not just lines, and shows what the model has captured.

Residuals are the distances between observed and predicted values; they show what the model has missed. `add_residuals()` adds them to the original data (residuals need the observed `y`, so a grid will not do). A residual plot against `x` should look like random noise if the model captured the pattern:

```
sim1 <- sim1 %>% add_residuals(sim1_mod)
ggplot(sim1, aes(resid)) + geom_freqpoly(binwidth = 0.5)
ggplot(sim1, aes(x, resid)) + geom_ref_line(h = 0) + geom_point()
```



13.4 Formulas and model families

Motivation. R's formula notation is a compact language for specifying model families. Seeing how a formula becomes a design matrix explains what each model can and cannot fit.

A formula `y ~ x` is translated into $y = a_1 + a_2 x$. `model_matrix()` shows the translation: the columns of the design matrix, one per coefficient, with an intercept column of ones unless `- 1` removes it.

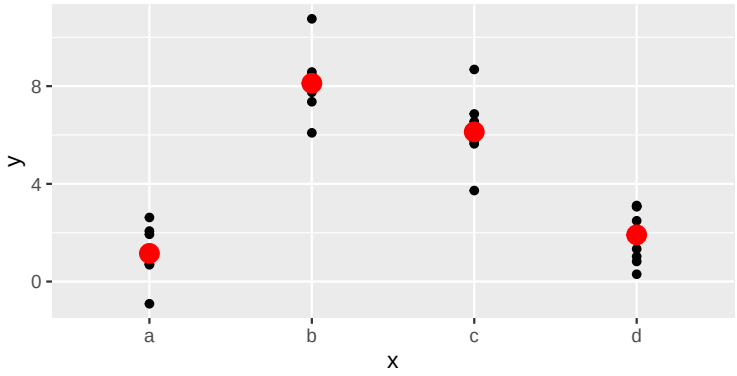
```
df <- tribble(~y, ~x1, ~x2,  4, 2, 5,  5, 1, 6)
model_matrix(df, y ~ x1 + x2)
```

```
## # A tibble: 2 x 3
##   `(Intercept)`    x1    x2
##           <dbl> <dbl> <dbl>
## 1             1     2     5
## 2             1     1     6
```

13.4.1 Categorical variables

For a categorical predictor `y ~ sex`, R creates an indicator column `sexmale` (1 for male, 0 otherwise); a column for `sexfemale` would be redundant, being one minus the other. With a categorical `x`, the fitted model predicts the mean of each category:

```
mod2 <- lm(y ~ x, data = sim2)
grid <- sim2 %>% data_grid(x) %>% add_predictions(mod2)
ggplot(sim2, aes(x)) +
  geom_point(aes(y = y)) +
  geom_point(data = grid, aes(y = pred), colour = "red", size = 4)
```



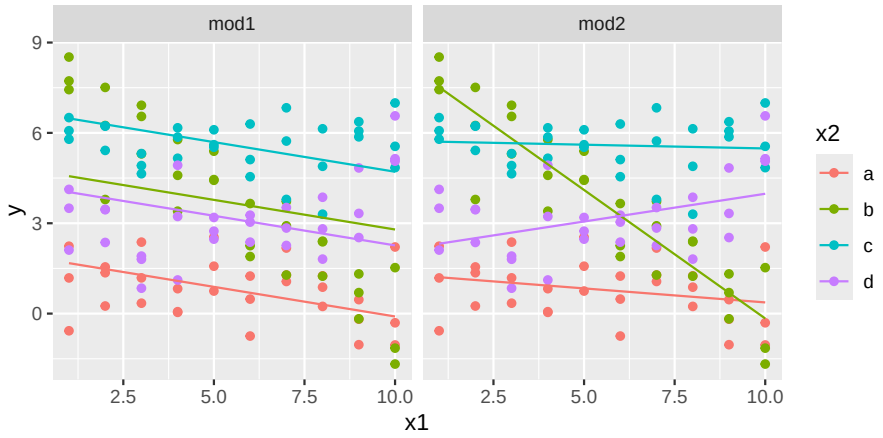
Predicting for a level the model never saw (`tibble(x = "e")`) is an error.

13.4.2 Interactions

`sim3` has a continuous `x1` and a categorical `x2`. Two models are possible: `y ~ x1 + x2` fits parallel lines (same slope, different intercepts), while `y ~ x1 * x2` also includes the interaction `x1:x2`, so each line has its own slope. `gather_predictions()` adds predictions from several models with a `model` column:

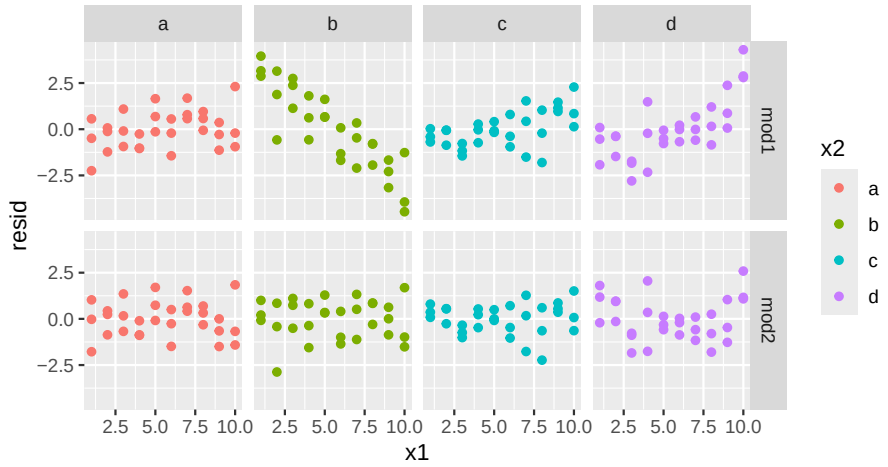
```
mod1 <- lm(y ~ x1 + x2, data = sim3)
mod2 <- lm(y ~ x1 * x2, data = sim3)
grid <- sim3 %>% data_grid(x1, x2) %>% gather_predictions(mod1, mod2)
ggplot(sim3, aes(x1, y, colour = x2)) +
  geom_point() +
```

```
  geom_line(data = grid, aes(y = pred)) +
  facet_wrap(~ model)
```



Which is better? The residuals of `mod1` still show clear patterns in groups `b`, `c`, `d`; those of `mod2` look random. In EDA, inspecting residuals answers the question; formal comparison is a topic of inference.

```
sim3 <- sim3 %>% gather_residuals(mod1, mod2)
ggplot(sim3, aes(x1, resid, colour = x2)) +
  geom_point() +
  facet_grid(model ~ x2)
```



With two continuous variables (`sim4`), `y ~ x1 * x2` fits $y = a_0 + a_1 x_1 + a_2 x_2 + a_{12} x_1 x_2$. A grid built with `seq_range(x1, 5)` (five evenly spaced values over the range) lets us compare the models; the interaction shows as slopes that change with the other variable.

13.4.3 Transformations

Transformations can be written inside the formula, e.g. `log(y) ~ sqrt(x1) + x2`. Operators that have a formula meaning (`+`, `*`, `^`, `-`) must be wrapped in `I()`: `y ~ x + I(x^2)` is the quadratic $a_1 + a_2 x + a_3 x^2$, whereas `y ~ x^2 + x` means `y ~ x`. Linear models can thus fit nonlinear curves, as long as the model is linear in the coefficients. `poly(x, 2)` gives orthogonal polynomials, and `splines::ns(x, df)` natural splines, which behave better outside the data range: polynomials diverge quickly beyond the data.

```
df <- tribble(~y, ~x,  1, 1,  2, 2,  3, 3)
model_matrix(df, y ~ x + I(x^2))
```

```
## # A tibble: 3 x 3
##   `(Intercept)`     x `I(x^2)`
##           <dbl> <dbl>    <dbl>
## 1             1     1        1
## 2             1     2        4
## 3             1     3        9
```

Extrapolation [Supplement]

Every model is fitted to the range of the data. Outside it the model's behaviour is dictated only by its family: a polynomial shoots to $\pm\infty$, a natural spline continues linearly. Neither is evidence about the data there, so predictions far outside the observed range should not be trusted.

13.5 Missing values and other model families

Rows with missing values cannot be used in the fit; `lm()` drops them silently by default, and with `na.action = na.warn` it warns. `nobs(mod)` gives the number of observations actually used.

Linear models assume a continuous residual with the same variance everywhere. Many extensions exist:

- generalised linear models, `glm()`, for binary or count responses;
- generalised additive models, `mgcv::gam()`, which fit smooth functions `y ~ s(x)`;
- penalised models such as `glmnet`, which shrink coefficients towards zero;
- robust models, `MASS::rlm()`, which reduce the influence of outliers;
- trees and forests (`rpart`, `randomForest`, `xgboost`).

They are fitted and explored with the same predictions-and-residuals approach.

Lecture 14. Model Building

14.1 Building models step by step

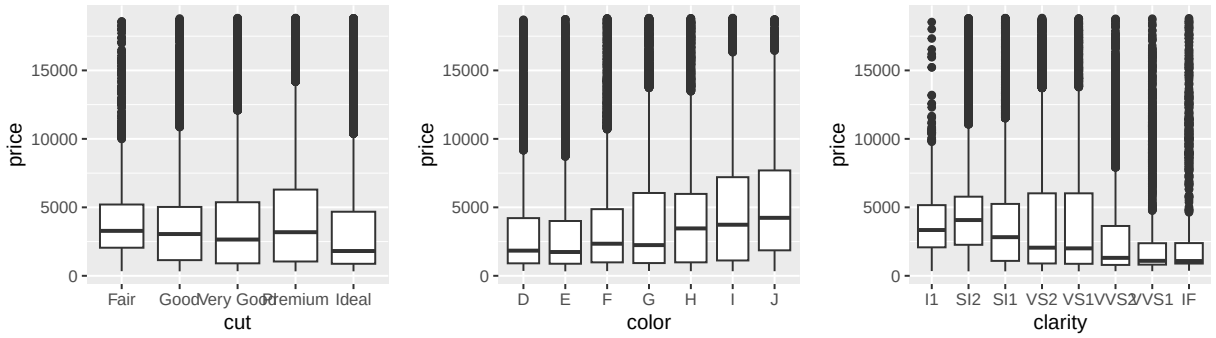
Motivation. Lecture 13 fitted models to simulated data with known structure. Real data are messier: several patterns overlap. The strategy here is to peel them off one at a time, fitting a model for the strongest pattern, looking at the residuals, and modelling what remains.

The workflow alternates between visualisation and modelling: find a pattern by plotting, make it precise with a model, then study the residuals for the next pattern. The aim is to move implicit knowledge ("prices depend on size") into an explicit, quantitative model. We use `modelr`, `nycflights13` and `lubridate`.

14.2 Why are low-quality diamonds more expensive?

Lecture 4 found that diamonds with poor cut, bad colour (J is the worst colour) and inferior clarity (I1) have higher median prices:

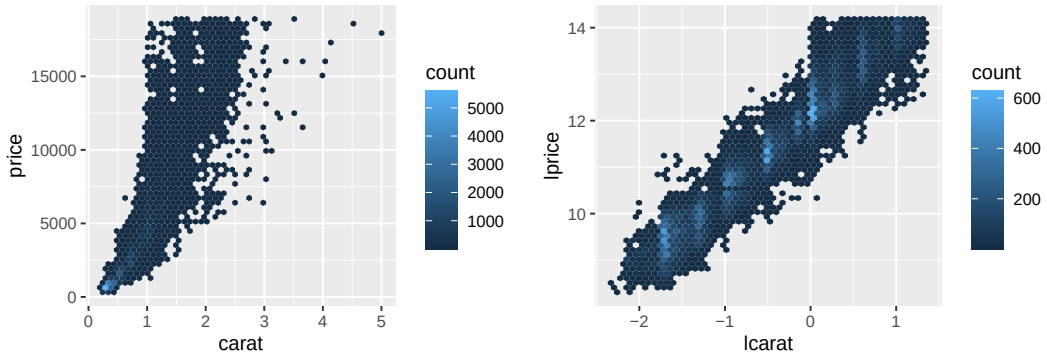
```
ggplot(diamonds, aes(cut, price)) + geom_boxplot()
ggplot(diamonds, aes(color, price)) + geom_boxplot()
ggplot(diamonds, aes(clarity, price)) + geom_boxplot()
```



14.2.1 Price and carat

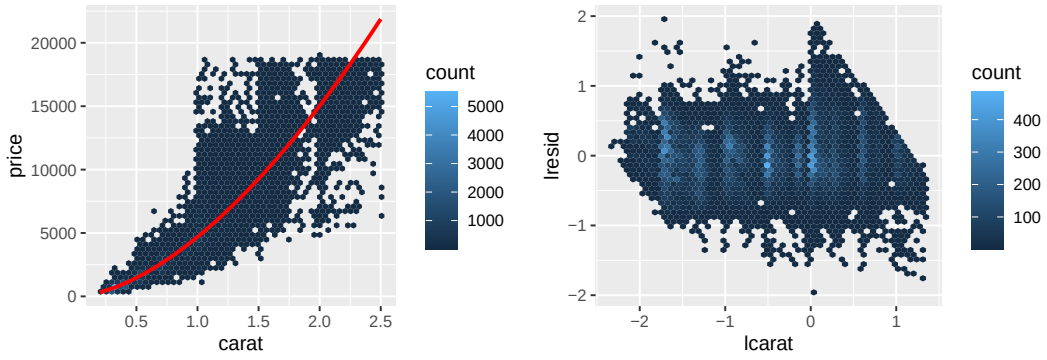
The explanation is a confounding variable: weight (`carat`) is the single most important factor in price, and low-quality diamonds tend to be larger. To see the effect of the other variables, first remove the effect of carat. Two adjustments make the relationship easier to model: keep diamonds of at most 2.5 carats (99.7% of the data) and log-transform both variables, which makes the pattern linear:

```
diamonds2 <- diamonds %>%
  filter(carat <= 2.5) %>%
  mutate(lprice = log2(price), lcarat = log2(carat))
ggplot(diamonds, aes(carat, price)) + geom_hex(bins = 50)
ggplot(diamonds2, aes(lcarat, lprice)) + geom_hex(bins = 50)
```



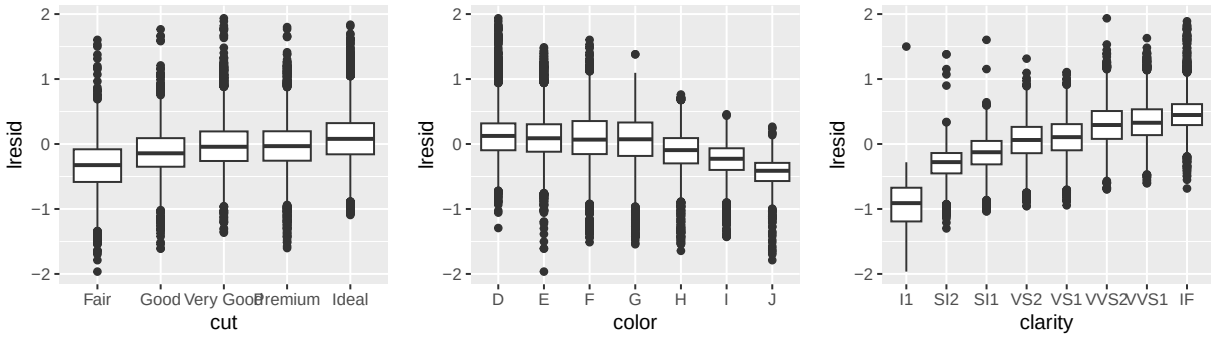
Fit a line on the log scale and back-transform the predictions to the original scale:

```
mod_diamond <- lm(lprice ~ lcarat, data = diamonds2)
grid <- diamonds2 %>%
  data_grid(carat = seq_range(carat, 20)) %>%
  mutate(lcarat = log2(carat)) %>%
  add_predictions(mod_diamond, "lprice") %>%
  mutate(price = 2 ^ lprice)
ggplot(diamonds2, aes(carat, price)) +
  geom_hex(bins = 50) +
  geom_line(data = grid, colour = "red", linewidth = 1)
diamonds2 <- diamonds2 %>% add_residuals(mod_diamond, "lresid")
ggplot(diamonds2, aes(lcarat, lresid)) + geom_hex(bins = 50)
```



The fitted curve shows that large diamonds are cheaper than the model predicts, probably because no diamond in the data costs more than $19 000. The residuals no longer depend on carat. Plotted against the quality variables, they now show the expected order:

```
ggplot(diamonds2, aes(cut, lresid)) + geom_boxplot()
ggplot(diamonds2, aes(color, lresid)) + geom_boxplot()
ggplot(diamonds2, aes(clarity, lresid)) + geom_boxplot()
```



Because the residuals are on the $\log_2$ scale, a residual of -1 means the price is half what carat alone predicts, and $+1$ means twice. As quality improves, the relative price increases.

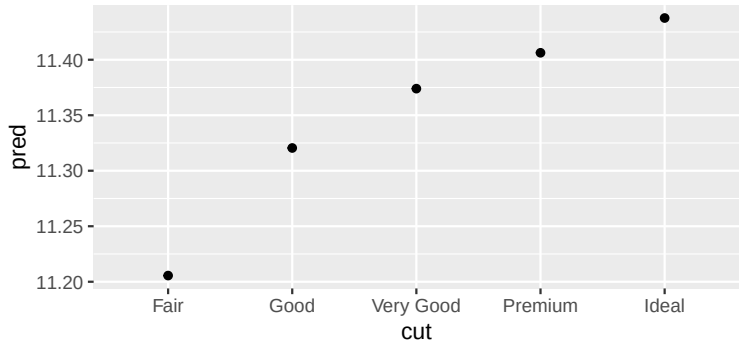
14.2.2 A more complicated model

Adding colour, cut and clarity makes their effects explicit:

```
mod_diamond2 <- lm(lprice ~ lcarat + color + cut + clarity, data = diamonds2)
```

With four predictors the model is hard to plot directly, but the effects can be plotted one at a time: `data_grid(cut, .model = mod_diamond2)` fills the other predictors with "typical" values (the median or most common level):

```
grid <- diamonds2 %>%
  data_grid(cut, .model = mod_diamond2) %>%
  add_predictions(mod_diamond2)
ggplot(grid, aes(cut, pred)) + geom_point()
```



The residuals of the larger model are mostly small, but some are large ($|r| > 1$, a factor of two). Looking at them is the start of a new exploration: they may be data-entry errors, or bargains.

```
diamonds2 <- diamonds2 %>% add_residuals(mod_diamond2, "lresid2")
diamonds2 %>%
  filter(abs(lresid2) > 1) %>%
  add_predictions(mod_diamond2) %>%
  mutate(pred = round(2 ^ pred)) %>%
  select(price, pred, carat:table, x:z) %>%
  arrange(price)
```

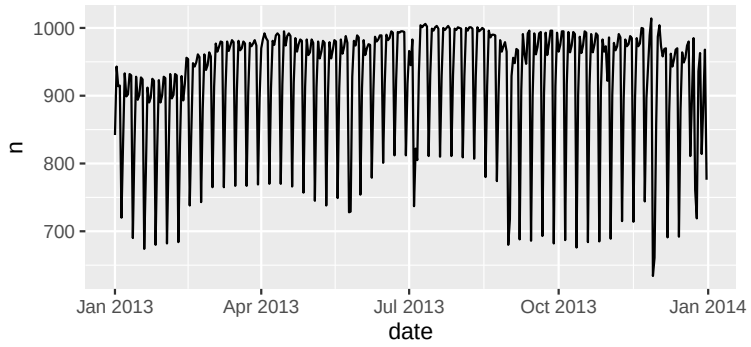
```
## # A tibble: 16 x 11
##   price  pred carat cut     color clarity depth table     x     y     z
##   <int> <dbl> <dbl> <ord>   <ord> <ord>   <dbl> <dbl> <dbl> <dbl> <dbl>
## 1  1013   264  0.25 Fair    F     SI2      54.4    64  4.3   4.23  2.32
## 2  1186   284  0.25 Premium G     SI2      59      60  5.33  5.28  3.12
## 3  1186   284  0.25 Premium G     SI2      58.8    60  5.33  5.28  3.12
## 4  1262  2644  1.03 Fair    E     I1       78.2    54  5.72  5.59  4.42
## 5  1415   639  0.35 Fair    G     VS2      65.9    54  5.57  5.53  3.66
## # i 11 more rows
```

14.3 What affects the number of daily flights?

Motivation. Counts over time mix several patterns: day-of-week cycles, seasons, holidays. Each layer is easier to see once the previous one has been modelled away.

Count the flights leaving NYC each day (365 rows):

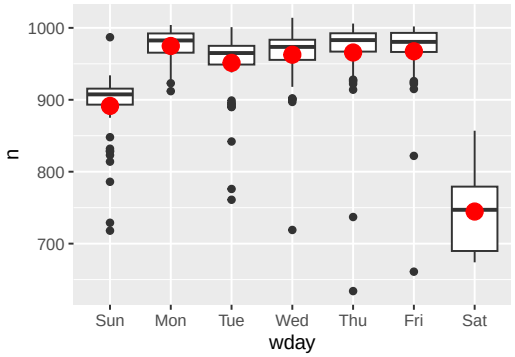
```
daily <- flights %>%
  mutate(date = make_date(year, month, day)) %>%
  group_by(date) %>%
  summarise(n = n())
ggplot(daily, aes(date, n)) + geom_line()
```

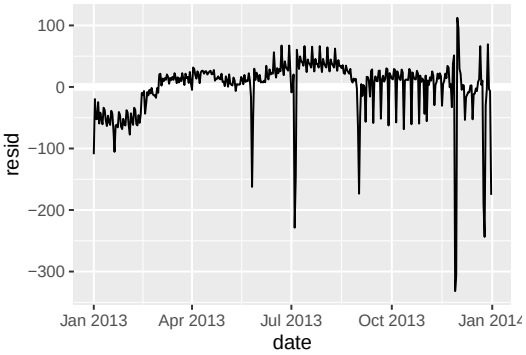


14.3.1 Day of week

The strongest pattern is weekly: most travel is for business, so there are fewer flights at weekends, especially Saturdays. A model with `wday` predicts the average per weekday:

```
daily <- daily %>% mutate(wday = wday(date, label = TRUE))
mod <- lm(n ~ wday, data = daily)
grid <- daily %>% data_grid(wday) %>% add_predictions(mod, "n")
ggplot(daily, aes(wday, n)) +
  geom_boxplot() +
  geom_point(data = grid, colour = "red", size = 4)
daily <- daily %>% add_residuals(mod)
daily %>%
  ggplot(aes(date, resid)) +
  geom_ref_line(h = 0) +
  geom_line()
```





The residuals reveal what the weekday model misses:

- from June the model fails, with a strong regular pattern left: colouring by weekday shows that Saturdays are badly fit (more Saturday flights in summer, fewer in autumn);
- some days have far fewer flights than expected; `filter(resid < -100)` identifies US holidays (New Year, 4 July, Thanksgiving, Christmas);
- a smooth long-term trend remains, fewer flights in January and December, more in summer (`geom_smooth(span = 0.20)`).

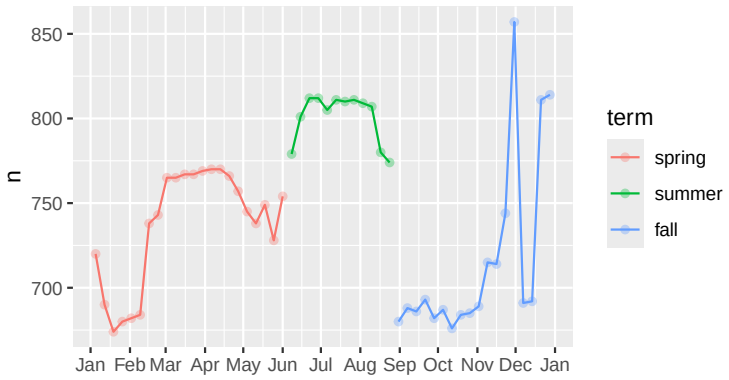
```
daily %>% filter(resid < -100)
```

```
## # A tibble: 11 x 4
##   date           n wday  resid
##   <date>     <int> <ord> <dbl>
## 1 2013-01-01   842 Tue   -109.
## 2 2013-01-20   786 Sun   -105.
## 3 2013-05-26   729 Sun   -162.
## 4 2013-07-04   737 Thu   -229.
## 5 2013-07-05   822 Fri   -145.
## # i 6 more rows
```

14.3.2 Seasonal Saturday effect

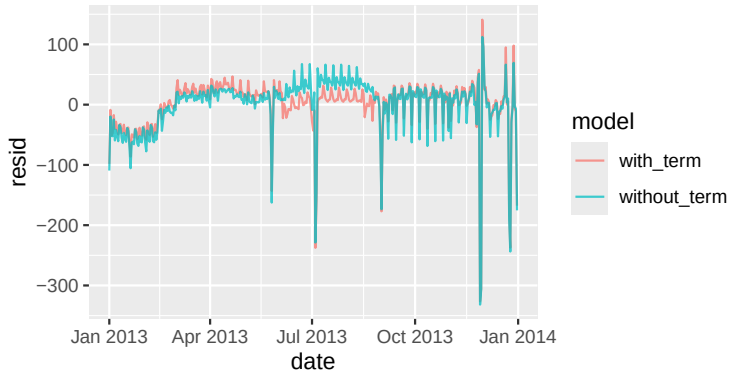
Plotting Saturdays alone suggests a school-term effect: people travel on Saturdays in summer holidays. A `term` variable with breaks chosen from the plot captures it:

```
term <- function(date) {
  cut(date,
      breaks = ymd(20130101, 20130605, 20130825, 20140101),
      labels = c("spring", "summer", "fall"))
}
daily <- daily %>% mutate(term = term(date))
daily %>%
  filter(wday == "Sat") %>%
  ggplot(aes(date, n, colour = term)) +
  geom_point(alpha = 1/3) +
  geom_line() +
  scale_x_date(NULL, date_breaks = "1 month", date_labels = "%b")
```



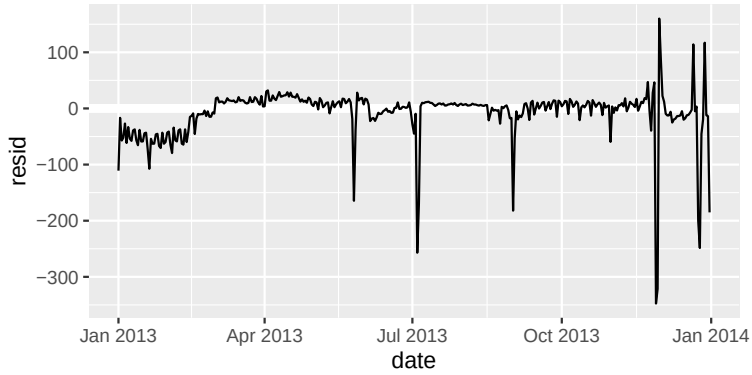
(The weekday labels depend on the locale: on a Korean system `wday(label = TRUE)` returns the Korean abbreviation instead, as on the course slides, so the filter must use that label.) Allowing the weekday effect to vary by term, `n ~ wday * term`, improves the fit:

```
mod1 <- lm(n ~ wday, data = daily)
mod2 <- lm(n ~ wday * term, data = daily)
daily %>%
  gather_residuals(without_term = mod1, with_term = mod2) %>%
  ggplot(aes(date, resid, colour = model)) +
  geom_line(alpha = 0.75)
```



The model fits the *mean* of each weekday–term cell, which is pulled by outliers (the holidays). A robust fit, `MASS::rlm()`, downweights outliers and gives a model that captures the regular pattern better, leaving the holidays clearly visible as large residuals:

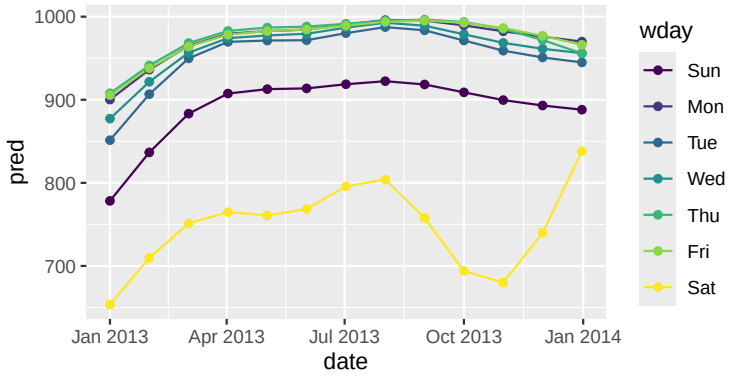
```
mod3 <- MASS::rlm(n ~ wday * term, data = daily)
daily %>%
  add_residuals(mod3, "resid") %>%
  ggplot(aes(date, resid)) +
  geom_hline(yintercept = 0, linewidth = 2, colour = "white") +
  geom_line()
```



14.3.3 Time of year: an alternative approach

Instead of encoding knowledge explicitly as terms, a more flexible model can let the data speak. A natural spline in date, interacting with weekday, fits a smooth seasonal curve for each weekday:

```
library(splines)
mod <- MASS::rlm(n ~ wday * ns(date, 5), data = daily)
daily %>%
  data_grid(wday, date = seq_range(date, n = 13)) %>%
  add_predictions(mod) %>%
  ggplot(aes(date, pred, colour = wday)) +
  geom_line() + geom_point()
```



The strong pattern in Saturday flights appears again without having been specified, which is reassuring: two different approaches pointing to the same signal.

Explicit versus flexible models [Supplement]

The `term` model encodes a hypothesis (school holidays) in three parameters per weekday and is easy to interpret, but its break dates were read off the plot, so it should not be tested on the same data. The spline model assumes only smoothness and uses more parameters; it is harder to interpret but less dependent on guesses. Agreement between them is evidence that the Saturday pattern is real rather than an artefact of either choice.